

Project Report

EstateMind

AI-Powered Intelligence for the Tunisian Real Estate Market



Private Higher School of Engineering and Technology – ESPRIT

Ministry of Higher Education and Scientific Research
Republic of Tunisia

Prepared by

Outliers

Farah Ben Yedder
Wael Ben Salem
Sirine Hjaïj
Mohamed Abdennadher
Zakaria Bouchaddakh
Aziz Hamlaoui

In collaboration with: VALUE



Supervised by: Mrs. Sarra Zouari and Mrs. Islem Maiti

Class: 4DS7

Academic Year: 2025 – 2026

Submission Date: May 13th, 2026



TABLE OF CONTENTS

LIST OF ABBREVIATIONS **viii**

1	Business Understanding	1
1.1	Introduction	1
1.2	Context and Motivation	1
1.3	Problem Statement	2
1.4	Business Objectives	2
1.4.1	BO1 — Centralize Real Estate Data	3
1.4.2	BO2 — Provide Data-Driven Property Price Insights	3
1.4.3	BO3 — Enable Detection of Market Anomalies and Investment Opportunities	3
1.4.4	BO4 — Analyze Regional Real Estate Market Trends	3
1.4.5	BO5 — Ensure Contract Compliance and Legal Safety	3
1.4.6	BO6 — Deliver Personalized Property Recommendations	3
1.5	Data Science Objectives	4
1.5.1	DSO1 — AI-Driven Data Ingestion and Processing	4
1.5.2	DSO2 — Property Price Estimation Model	4
1.5.3	DSO3 — Market Anomaly Detection Models	4
1.5.4	DSO4 — Temporal and Regional Trend Modeling	4
1.5.5	DSO5 — Automated Contract Analysis and Compliance Verification	4
1.5.6	DSO6 — Personalized Recommendation System	5
1.6	Stakeholders	5
1.7	Business Success Criteria	5
1.8	Project Scope and Constraints	6
1.8.1	In Scope	6
1.8.2	Out of Scope	6
1.8.3	Constraints	6
1.9	Alignment with Sustainable Development Goals	6
1.10	Conclusion	7

2	Data Acquisition and Understanding	8
2.1	Introduction	8
2.2	Data Sources	8
2.2.1	Legal Text Data Sources	10
2.3	Web Scraping Strategy	11
2.4	Data Collection Pipeline	12
2.4.1	End-to-End Workflow	12
2.4.2	MASTER_SCHEMA and Standardization Logic	13
2.4.3	Source-to-Schema Mapping	14
2.4.4	Unstructured Content Preservation for NLP and Vision Pipelines	15
2.4.5	Legal Corpus Collection Pipeline	15
2.5	Raw Dataset Description	16
2.5.1	Dataset Structure and Schema Overview	17
2.5.2	Source Coverage	17
2.5.3	Sample Records	18
2.5.4	Initial Field Availability	18
2.6	Data Quality Assessment	18
2.6.1	Missing Values Analysis	19
2.6.2	Numeric Feature Distributions	19
2.6.3	Categorical Feature Analysis	20
2.6.4	Price Variability Across Categories	21
2.7	Data Preparation and Normalization	22
2.7.1	Deduplication	22
2.7.2	Invalid Value Filtering	23
2.7.3	Categorical and Text Normalization	23
2.7.4	Feature Engineering	23
2.7.5	Modeling Readiness Validation	24
2.8	Challenges and Limitations	24
2.8.1	Data Fragmentation and Platform Heterogeneity	24
2.8.2	Duplicate Listings Across Platforms	25
2.8.3	Asking Prices vs. Transaction Prices	25
2.8.4	Missing and Incomplete Attributes	25
2.8.5	Unstructured Text Complexity	26
2.8.6	Legal Text Accessibility and Quality	26
2.8.7	Temporal Coverage Constraints	26
2.8.8	Geographic Bias	26

2.9	Preliminary Insights	27
2.9.1	Temporal Patterns	27
2.9.2	Geographic Distribution	27
2.9.3	Geospatial Visualization	28
2.9.4	Correlation and Structural Relationships	29
2.10	Conclusion	29
3	Modeling	31
3.1	Introduction	31
3.2	Global System Architecture	32
3.3	BO1: Data Unification and Structuring	32
3.3.1	Business Objective	32
3.3.2	Data Science Objective	33
3.3.3	Modeling Approach	33
3.3.4	Implementation Overview	35
3.4	BO2: NLP and Vision Feature Extraction	38
3.4.1	Business Objective	38
3.4.2	Data Science Objective	39
3.4.3	Modeling Approach	39
3.4.3.1	NLP Extraction Pipeline	39
3.4.3.2	Vision Extraction Pipeline	41
3.4.4	Implementation Overview	42
3.5	BO3: RAG System for Statistical and Economic Documents	43
3.5.1	Business Objective	43
3.5.2	Data Science Objective	43
3.5.3	Modeling Approach	43
3.5.3.1	Document Ingestion and Chunking	43
3.5.3.2	NLP Enrichment - Keyword Extraction	44
3.5.3.3	Embedding and Vector Indexing	45
3.5.3.4	Retrieval with Reranking	46
3.5.3.5	Answer Generation	47
3.5.4	Implementation Overview	47
3.6	BO4: Price Estimation Modeling	48
3.6.1	Business Objective	48
3.6.2	Data Science Objective	48
3.6.3	Modeling Approach	49

3.6.3.1	Feature Engineering	49
3.6.3.2	Model Selection	49
3.6.3.3	Evaluation Metrics	50
3.6.3.4	Hyperparameter Optimization	51
3.6.4	Implementation Overview	51
3.6.4.1	Results	51
3.6.4.2	Model Selection Criteria	52
3.6.4.3	Selected Model	53
3.7	BO5: Price Deviation Detection	54
3.7.1	Business Objective	54
3.7.2	Data Science Objective	55
3.7.3	Modeling Approach	55
3.7.3.1	Price Gap Computation	55
3.7.3.2	Multi-Method Anomaly Scoring	56
3.7.3.3	Model Benchmarking	57
3.7.3.4	Opportunity Labelling	57
3.7.3.5	Confidence Score	58
3.7.4	Implementation Overview	59
3.7.4.1	Pipeline Summary	59
3.7.4.2	Anomaly Detection Improvements	60
3.7.4.3	Explainability	60
3.7.4.4	Geospatial Output	62
3.8	BO6: Market Trend Analysis	62
3.8.1	Business Objective	62
3.8.2	Data Science Objective	62
3.8.3	Modeling Approach	62
3.8.3.1	Feature Engineering (Gold+ Table)	62
3.8.3.2	True Time-Series Forecasting	63
3.8.4	Implementation Overview	64
3.9	BO7: Legal Compliance Analysis	64
3.9.1	Business Objective	64
3.9.2	Data Science Objective	65
3.9.3	Modeling Approach	65
3.9.3.1	Data Collection and Chunking	65
3.9.3.2	NLP Enrichment — Keyword Extraction	65
3.9.3.3	Embedding and Vector Indexing	66

3.9.3.4	Retrieval with Reranking	66
3.9.3.5	Retrieval Evaluation	66
3.9.3.6	Answer Generation	67
3.9.3.7	Generation Evaluation — LLM-as-a-Judge	67
3.9.4	Implementation Overview	67
3.10	BO8: Recommendation System	68
3.10.1	Business and Data Science Objective	68
3.10.2	Justification of the Approach	68
3.10.3	System Architecture Overview	69
3.10.4	Query Classification	70
3.10.5	Query Understanding	70
3.10.6	Business Filters	70
3.10.7	Ranking Engine	70
3.10.8	Explainability	70
3.10.9	Response Generator	71
3.10.10	Recommendation Strategy and Evaluation	71
3.11	Conclusion	72
4	Deployment	73
4.1	Introduction	73
4.2	Global Architecture	73
4.3	Public-Facing Application	75
4.3.1	Technology Stack	75
4.3.2	AI Agent Integration	76
4.3.2.1	Price Estimation Agent	76
4.3.2.2	Anomaly Detection Agent	76
4.3.2.3	Law Agent	76
4.3.2.4	Legal Analysis Result Presentation	77
4.3.2.5	Recommendation Agent	78
4.3.2.6	Image Captioning Service	79
4.4	Administration Dashboard	79
4.4.1	Overview	79
4.4.2	Internationalization	80
4.4.3	Interactive Map	80
4.4.4	Property Image Handling	81
4.5	Backend API	81

TABLE OF CONTENTS

4.5.1	Overview	81
4.5.2	Performance Optimizations	81
4.5.2.1	Database Indexing	81
4.5.2.2	Cache Warming	82
4.5.2.3	Query Optimization	82
4.5.3	Service Authentication	82
4.6	Access Control and Security	83
4.6.1	Role-Based Access	83
4.6.2	Environment Isolation	83
4.7	Key Engineering Decisions	83
4.8	Conclusion	84
GENERAL CONCLUSION		85
APPENDIX: LinkedIn Posts		87
BIBLIOGRAPHY		91



GENERAL INTRODUCTION

The real estate market plays an important role in economic development, urban planning, and social stability. In Tunisia, this market is characterized by strong regional disparities, fragmented data sources, and limited transparency in pricing and decision-making. As a result, market actors such as agencies, buyers, sellers, and investors often rely on incomplete or subjective information.

The EstateMind project aims to address these challenges by proposing a data-driven and intelligent decision-support approach for the Tunisian real estate market. By leveraging data engineering, machine learning, and artificial intelligence techniques, EstateMind seeks to transform heterogeneous and unstructured real estate data into meaningful insights that support fair pricing, market analysis, and informed decision-making.

This project follows the Team Data Science Process (TDSP) methodology, which provides a structured framework for designing data science solutions aligned with real business needs. The report is organized into four main chapters, corresponding to the key TDSP phases:

- **Business Understanding**
- **Data Acquisition and Understanding**
- **Modeling**
- **Deployment**

Each chapter addresses a specific phase of the TDSP methodology and contributes to the progressive development of the EstateMind solution, from problem definition to model assessment.



LIST OF ABBREVIATIONS

- **AI** – Artificial Intelligence
- **ML** – Machine Learning
- **TDSP** – Team Data Science Process
- **BO** – Business Objective
- **DSO** – Data Science Objective
- **NLP** – Natural Language Processing
- **API** – Application Programming Interface
- **URL** – Uniform Resource Locator
- **JSON** – JavaScript Object Notation
- **CSV** – Comma-Separated Values
- **GPS** – Global Positioning System
- **MASTER_SCHEMA** – Unified Reference Data Structure for Listing Normalization

Chapter

1

Business Understanding

1.1 Introduction

This chapter presents the Business Understanding phase of the EstateMind project following the TDSP methodology. It introduces the Tunisian real estate context, identifies key market challenges, and defines the business and data science objectives that guide the rest of the project.

1.2 Context and Motivation

The real estate sector represents a fundamental component of economic development, urban planning, and social stability. In Tunisia, real estate activities influence household wealth, private investment, infrastructure development, and regional attractiveness. Housing affordability, urban expansion, and regional disparities are closely linked to the dynamics of the real estate market.

Despite its strategic importance, the Tunisian real estate market remains largely informal and opaque. Property prices vary significantly across regions and time periods, yet pricing decisions are often based on subjective judgment, personal experience, or informal negotiation rather than objective market indicators. Moreover, real estate data is fragmented across online platforms, agencies, and informal channels, making it difficult to obtain a comprehensive and reliable view of the market.

The increasing availability of digital real estate listings and publicly accessible socio-economic indicators offers a unique opportunity to modernize this sector. By leveraging Artificial Intelligence (AI), Machine Learning (ML), and data engineering techniques, it becomes possible to transform

heterogeneous and unstructured data into meaningful insights. These technologies can support fair pricing, identify market inefficiencies, and improve transparency for all market participants.

EstateMind is motivated by the need to bridge the gap between raw real estate data and actionable decision support tailored to the Tunisian context.

1.3 Problem Statement

Although large volumes of real estate data are generated daily, stakeholders face significant challenges that limit effective market analysis and informed decision-making:

- Absence of a centralized, structured, and trustworthy real estate data repository
- Difficulty in estimating fair and objective property prices
- Limited visibility into underpriced and overpriced properties
- Insufficient analysis of regional and temporal market dynamics
- Weak integration of socio-economic factors influencing real estate values
- Manual, time-consuming, and error-prone verification of real estate contract compliance

As a result, buyers, sellers, investors, and professionals often rely on incomplete or biased information, increasing financial risk, reducing market efficiency, and reinforcing regional inequalities.

1.4 Business Objectives

The primary goal of this project is to develop **EstateMind**, an intelligent decision-support system dedicated to the Tunisian real estate market. The system aims to improve transparency, efficiency, and trust by addressing the following business objectives:

1.4.1 BO1 — Centralize Real Estate Data

Enable a single, reliable source of truth by consolidating fragmented real estate data from multiple heterogeneous sources, improving data accessibility, quality, and consistency.

1.4.2 BO2 — Provide Data-Driven Property Price Insights

Support buyers and sellers with AI-based property price estimation to reduce information asymmetry and promote fair and transparent pricing.

1.4.3 BO3 — Enable Detection of Market Anomalies and Investment Opportunities

Assist investors in identifying underpriced and overpriced properties by detecting deviations from prevailing market patterns, highlighting potential opportunities and valuation risks.

1.4.4 BO4 — Analyze Regional Real Estate Market Trends

Provide insights into how real estate prices and market dynamics evolve across Tunisian regions over time, identifying growth areas, stagnation zones, and emerging hotspots.

1.4.5 BO5 — Ensure Contract Compliance and Legal Safety

Reduce legal risks by supporting the verification of real estate contracts against Tunisian regulatory requirements, ensuring secure and compliant transactions.

1.4.6 BO6 — Deliver Personalized Property Recommendations

Enhance user experience by providing personalized property recommendations tailored to user preferences, behavioral patterns, and contextual constraints.

This objective aims to reduce search time, improve decision quality, and support buyers and investors in identifying properties aligned with their specific needs.

1.5 Data Science Objectives

To achieve the defined business objectives, the following data science objectives are established:

1.5.1 DSO1 — AI-Driven Data Ingestion and Processing

Develop an intelligent data ingestion agent capable of collecting, cleaning, normalizing, and structuring real estate data from heterogeneous sources into a unified analytical dataset.

1.5.2 DSO2 — Property Price Estimation Model

Design, train, and validate machine learning regression models to estimate property prices based on intrinsic attributes (location, size, property type) and external socio-economic indicators.

1.5.3 DSO3 — Market Anomaly Detection Models

Develop anomaly detection models to identify significant discrepancies between observed property prices and expected market values, with explainable indicators for decision support.

1.5.4 DSO4 — Temporal and Regional Trend Modeling

Apply time-series analysis and trend-based clustering to model regional price evolution and identify similar market trajectories across Tunisia.

1.5.5 DSO5 — Automated Contract Analysis and Compliance Verification

Implement NLP and rule-based techniques to extract key clauses from real estate contracts and verify compliance with Tunisian legal requirements.

1.5.6 DSO6 — Personalized Recommendation System

Design and implement a hybrid recommendation system that combines content-based filtering and behavioral interaction analysis to suggest relevant properties to users. The system will leverage structured property features and user interaction data to generate ranked recommendations and evaluate performance using appropriate ranking metrics.

1.6 Stakeholders

EstateMind targets a diverse set of stakeholders with distinct needs:

- **Buyers:** Identify fairly priced properties and reduce the risk of overvaluation
- **Sellers:** Price properties competitively based on objective market insights
- **Investors:** Detect investment opportunities and valuation anomalies
- **Real Estate Agencies:** Enhance valuation accuracy and advisory services
- **Regulatory Bodies:** Improve market transparency and compliance monitoring

1.7 Business Success Criteria

The success of EstateMind will be evaluated based on the following criteria:

- Improved accuracy of AI-based property price estimations
- Effective identification of underpriced and overpriced properties
- Clear, explainable, and interpretable analytical outputs
- Adoption and usability across multiple stakeholder groups
- Increased transparency and trust in market analysis

1.8 Project Scope and Constraints

1.8.1 In Scope

- Residential and commercial real estate analysis in Tunisia
- Historical and current real estate market data
- AI-driven analytics, intelligent agents, and decision-support dashboards

1.8.2 Out of Scope

- Automated legal decision-making or contract approval
- Real-time financial or transactional systems

1.8.3 Constraints

- Variability and incompleteness of real estate data
- Limited availability of official datasets
- Regulatory and privacy considerations related to data usage

1.9 Alignment with Sustainable Development Goals

EstateMind contributes to several United Nations Sustainable Development Goals (SDGs):

- **SDG 11 — Sustainable Cities and Communities:** Supports urban planning and regional analysis through transparent real estate insights.
- **SDG 10 — Reduced Inequalities:** Highlights regional and socio-economic disparities using objective data-driven indicators.

- **SDG 9 — Industry, Innovation, and Infrastructure:** Introduces AI-driven innovation into a traditionally informal sector.
- **SDG 12 — Responsible Consumption and Production:** Encourages informed and responsible real estate investment decisions.



1.10 Conclusion

This chapter clarified the main challenges of the Tunisian real estate market and translated them into well-defined business and data science objectives. These outcomes establish a clear foundation for the subsequent Data Understanding and Data Preparation phases.

Data Acquisition and Understanding

2.1 Introduction

This chapter presents the Data Acquisition, Understanding, and Preparation phase of the EstateMind project, following the Microsoft Team Data Science Process (TDSP). The objective of this phase is to collect real estate data from heterogeneous online sources, analyze its structure and quality, and transform it into a structured dataset suitable for analytical modeling.

Given the fragmented nature of the Tunisian real estate market, automated scraping and preprocessing techniques are required to convert raw and semi-structured web content into consistent and reliable data. This chapter describes the selected data sources, the scraping methodology, the cleaning and normalization processes, and the structure of the resulting analytical dataset.

The outcomes of this phase establish the data foundation required for predictive modeling, anomaly detection, trend analysis, NLP and vision feature extraction, legal compliance analysis, and recommendation system development.

2.2 Data Sources

For the EstateMind project, we selected a set of major Tunisian real estate listing platforms that provide comprehensive coverage of both residential and commercial properties, represent listings from agencies as well as individuals, and include structured attributes necessary for analysis. These platforms collectively cover multiple regions across Tunisia and offer detailed property information including price, location, surface area, number of rooms, and other features.

The selected sources are: Mubawab Tunisie, Immobilier.com.tn, Tayara.tn, Tunisie Annonce, Menzili.tn, TunisiaHouse.tn, TunisiaPromo.com, Bigdatis.tn, DarcomTunisia.com, Monbien.tn, and OpenScoop.com.

From these platforms, the data collected for each property listing includes the following attributes:

- **Basic Listing Information:** title (titre), description (description), URL (url), source type (agency or individual), and publication date when available
- **Location:** full address (adresse), city (ville), region/governorate (region), latitude and longitude (latitude, longitude)
- **Property Characteristics:** price (prix), surface area (surface), number of rooms (pieces), property type (type), contract type (contrat), floor (etage), construction year (annee_constr), and various amenities and features such as air conditioning, elevator, heating, garden, parking, balcony, security, and more
- **Proximity Features:** distances or presence of schools (ecole), pharmacies (pharmacie), hospitals (hopital), markets (marche), shops (magasin), restaurants and bars (restaurant, bar), and public transportation (transport)
- **Multimedia:** images associated with each property (images)

The inclusion of multiple sources was necessary due to the lack of a centralized official real estate database in Tunisia. Prices for similar properties often vary significantly between platforms, and listings from a single platform may not cover all geographic regions or market segments. By aggregating data from these diverse sources, we reduce potential bias, improve coverage across cities and neighborhoods, and strengthen the project's first business objective (BO1) of data centralization.

Beyond structured fields, a significant portion of the analytical value in this dataset lies in **unstructured content**. Free-text listing descriptions — written in French, Arabic, or Tunisian dialect — and property image galleries constitute the primary input for the NLP and Vision

feature extraction pipeline (BO2). These unstructured fields are systematically collected alongside structured attributes and preserved in raw form to enable downstream extraction of missing attributes such as surface area, number of rooms, amenities, and visual property features.

Despite these advantages, several limitations are inherent in the selected sources. Duplicate listings across platforms are frequent, price formats are inconsistent, some attributes are missing or partially filled, and descriptive fields are unstructured, requiring extensive preprocessing. Additionally, only asking prices are available, with no access to official transaction records. Recognizing these limitations is essential for understanding data quality and guiding preprocessing and normalization steps in the subsequent phase.

2.2.1 Legal Text Data Sources

In addition to real estate listing data, a second distinct data collection effort was conducted to support the Legal Compliance Analysis (BO6) and the RAG System for Tunisian Legal Codes (BO7). These modules require access to structured legal text rather than property listings.

Legal articles were acquired from two complementary sources. The primary source is **9anoun.tn**, a Tunisian legal information platform that publishes the full text of national legislation article by article. Three legal codes were targeted: the *Code des Obligations et des Contrats* (COC), the *Code des Droits Réels* (CDR), and the *Code de l'Aménagement du Territoire et de l'Urbanisme* (CATU). Each article was retrieved individually and stored as a structured record containing the article number, full text, and source law metadata.

The secondary source consists of **PDF-based legal documents** for codes not available in a web-parseable format. These documents were processed using a document conversion pipeline capable of handling both digital and scanned PDFs, producing structured markdown output that was subsequently cleaned and segmented at the article level.

The resulting legal corpus comprised **1,116 article-level chunks** across the three targeted legal codes, stored in JSONL format for downstream embedding and retrieval.

2.3 Web Scraping Strategy

To acquire property data from multiple online sources, we evaluated several popular web scraping tools and frameworks based on their capabilities, ease of use, and suitability for the Tunisian real estate market. Among the tools considered were Requests combined with BeautifulSoup, Scrapy, Selenium, API scraping using requests and JSON, and asynchronous scraping with httpx and asyncio. Requests + BeautifulSoup is lightweight and well-suited for simple HTML parsing, whereas API scraping allows direct retrieval of structured data from backend endpoints when available. Async scraping offers high-performance parallel data retrieval, but requires more complex implementation.

Following this evaluation, we selected a hybrid approach combining **Scrapy**, **Selenium**, and **BeautifulSoup** to handle the diversity of the targeted websites. Scrapy serves as the main framework for large-scale crawling of static pages and structured listings, offering built-in support for pipelines and asynchronous requests. Selenium is used for pages requiring user interactions, such as logins, cookie consent, or CAPTCHA, and for dynamically rendered content. BeautifulSoup is employed for fine-grained HTML parsing where Scrapy selectors or Selenium DOM traversal alone are insufficient, particularly for irregular or deeply nested structures.

For the legal corpus (BO7), a dedicated scraping module was implemented using **Requests** and **BeautifulSoup** to extract articles from the 9anoun.tn platform, targeting the prose CSS class container and applying rate limiting to ensure respectful crawling. For PDF-based legal codes, **Docling** was used as the document ingestion framework, with adaptive strategy selection between standard text extraction and full-page OCR depending on the document format.

This combination provides the flexibility to manage heterogeneous website structures, dynamic content, multi-step navigation flows, and PDF-based legal documents, ensuring reliable data extraction while maintaining data quality and consistency across all collected datasets.

2.4 Data Collection Pipeline

After identifying the target platforms and selecting an appropriate scraping stack, we designed an end-to-end data collection pipeline to ensure that heterogeneous listings could be extracted, standardized, and consolidated into a unified analytical dataset. The pipeline follows a multi-stage workflow, from raw web pages to a normalized dataset compliant with a master schema.

2.4.1 End-to-End Workflow

The acquisition workflow is organized into the following stages:

1. **Source Crawling and Page Discovery:** each platform is crawled to discover listing URLs and multi-page pagination endpoints. For static websites, Scrapy handles large-scale crawling efficiently; for dynamic pages, Selenium is used to render the content before extraction.
2. **Structured Extraction (Per-Source Parsers):** each source has a dedicated extractor that retrieves the main fields available on the listing page (e.g., price, surface, location, property type) along with unstructured fields (title, description) and multimedia links (images).
3. **Raw Data Storage (Bronze Layer):** extracted records are stored in source-specific raw outputs (JSON/CSV). This step ensures traceability and allows re-processing without re-scraping the websites.
4. **Schema Normalization (Silver Layer):** raw records are mapped to a unified **MASTER_SCHEMA**, converting inconsistent formats into standardized representations (numeric parsing, unit harmonization, date normalization, categorical standardization).
5. **Dataset Consolidation (Gold Layer):** normalized data from all sources are concatenated into a single dataset. Duplicates and invalid rows are flagged for downstream cleaning and quality checks.

Figure 2.1 illustrates the complete pipeline used in EstateMind.

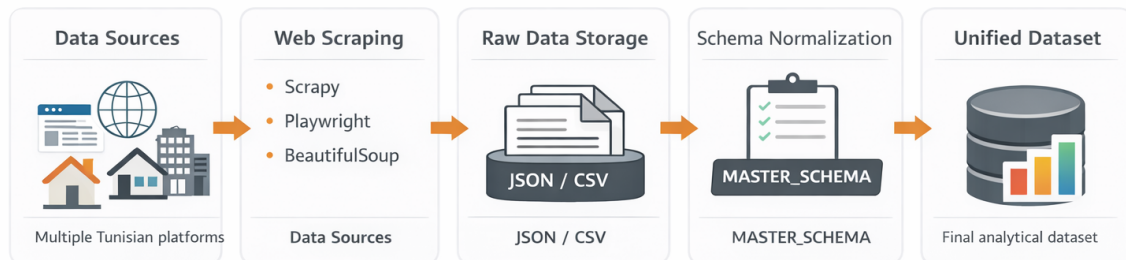


Figure 2.1: EstateMind end-to-end data collection pipeline from multi-source scraping to schema normalization and unified dataset export.

2.4.2 MASTER_SCHEMA and Standardization Logic

Given the diversity of Tunisian real estate platforms, field names, formats, and availability vary significantly across sources. To ensure consistency, we defined a centralized **MASTER_SCHEMA** that acts as the reference structure for all listings (e.g., prix, surface, ville, region, contrat, type, pieces, images). Each source is mapped to this schema through explicit transformation rules.

To improve reproducibility and auditability, the standardization step includes:

- **Numeric parsing:** converting strings such as “1,200 TND” into numeric values and removing separators or currency tokens.
- **Unit harmonization:** ensuring surfaces are consistently represented in square meters.
- **Date normalization:** parsing publication dates into a consistent format when available.

- **Categorical standardization:** normalizing values for `contrat` (sale/rent), `type` (apartment/house/land) and geographic fields (city/governorate naming consistency).

Table 2.1 presents an excerpt of the `MASTER_SCHEMA` used for normalization.

Table 2.1: Excerpt of the unified `MASTER_SCHEMA` used to standardize heterogeneous property listings.

Field	Description
<code>listing_id</code>	Unique internal identifier for each listing
<code>source</code>	Platform name (e.g., Tayara, Mubawab, Aqari, etc.)
<code>url</code>	Direct URL of the original listing
<code>titre</code>	Original listing title as published
<code>description</code>	Unstructured textual description
<code>region</code>	Standardized governorate/region field
<code>ville</code>	Standardized city field
<code>prix</code>	Listing asking price (numeric, TND)
<code>surface</code>	Area in square meters (numeric)
<code>type</code>	Standardized property type (apartment/house/land, etc.)
<code>contrat</code>	Transaction type (sale/rent)
<code>pieces</code>	Number of rooms when available or inferred
<code>images</code>	List of image URLs linked to the listing

2.4.3 Source-to-Schema Mapping

For each platform, a mapping layer converts source-specific fields into the `MASTER_SCHEMA`. This step is critical because some websites provide structured fields explicitly (e.g., separate tags for surface and rooms), whereas others embed the same information in unstructured descriptions.

Table 2.2 shows an illustrative mapping example for two sources.

Table 2.2: Illustrative example of source-to-master schema mapping.

MASTER_SCHEMA field	Source A field	Source B field
prix	price_value	prix_annonce
surface	area_m2	extracted from details_text
ville	city	localite
region	inferred from address	gouvernorat
pieces	rooms	extracted via regex from description

2.4.4 Unstructured Content Preservation for NLP and Vision Pipelines

A dedicated preservation strategy was applied to the two unstructured data modalities required by the NLP and Vision Feature Extraction pipeline (BO2).

Free-text descriptions (`description` and `titre`) are retained in their original form throughout the Bronze and Silver layers without truncation or normalization of content. This ensures that the full linguistic signal — including multilingual constructs in French, Arabic, and Tunisian dialect — remains available for transformer-based attribute extraction in the modeling phase.

Property image URLs are collected and stored as lists in the `images` field. In the modeling phase, these URLs are resolved and images are downloaded using a concurrent thread pool, cached locally, and fed into vision model pipelines for zero-shot amenity detection. The decision to store URLs rather than raw images at the acquisition stage reduces storage overhead while preserving retrieval flexibility.

2.4.5 Legal Corpus Collection Pipeline

The legal data collection pipeline, designed to support BO7, operates independently from the real estate listing pipeline and follows a dedicated acquisition and structuring workflow.

For web-based legal sources, a **BeautifulSoup**-based scraper was implemented to extract articles from `9anoun.tn`. The scraper targeted the prose CSS class container on each article page, removed tooltip metadata spans, and applied rate limiting at 0.3 seconds per request to ensure respectful and stable crawling. Each extracted article was saved as a structured JSONL record containing the fields `id`, `article_number`, `text`, and a metadata dictionary specifying the source law name.

For PDF-based legal codes, a document conversion pipeline was constructed using **Docling** with adaptive strategy selection: standard text extraction was applied to digital PDFs, while full-page OCR via `PdfPipelineOptions` was used for scanned or locked documents. The converted markdown output was cleaned using a multi-step regex pipeline that removed table of contents lines, boilerplate phrases, Arabic tatweel characters, and HTML image placeholders. Structural markers (`Article`, `Titre`, `Chapitre`, `Section`, `Livre`) were used as chunk boundaries.

The resulting legal corpus comprised **1,116 article-level chunks** across three Tunisian legal codes, stored in `scraped_enriched_data.jsonl` and used as the document collection for embedding and semantic retrieval in BO7.

Overall, this pipeline ensures that EstateMind can continuously ingest heterogeneous listings while preserving traceability (raw storage), improving consistency (master schema), and preparing a consolidated dataset suitable for quality assessment and downstream modeling.

2.5 Raw Dataset Description

This section provides an overview of the raw dataset obtained after multi-source scraping and schema normalization. The objective is to describe the dataset structure, highlight its multi-platform nature, and summarize the main variables before applying advanced cleaning and quality correction procedures.

2.5.1 Dataset Structure and Schema Overview

All collected listings are consolidated into a unified tabular format aligned with the MASTER_SCHEMA. Each row represents one property listing, identified by a unique internal identifier (`listing_id`) and linked to its original source platform (`source`) and URL (`url`). The dataset contains a combination of:

- **Textual attributes:** `titre`, `description`, `adresse`
- **Geographic attributes:** `ville`, `region`, `latitude`, `longitude`
- **Numeric attributes:** `prix`, `surface`, `pieces`, `etage`, `annee_constr`
- **Categorical attributes:** `type`, `contrat`, plus optional amenities and proximity-related fields
- **Multimedia links:** `images` (one or multiple URLs)

2.5.2 Source Coverage

Since the Tunisian real estate market is fragmented, collecting data from multiple platforms is essential to reduce platform-specific bias and improve coverage across cities and regions. The distribution of listings per source is summarized in Figure ??.

The consolidated dataset integrates listings collected from multiple Tunisian platforms. The volume of observations varies across sources due to differences in platform size, pagination depth, and scraping constraints. Larger national platforms contribute a higher number of listings, while smaller or regionally focused platforms provide complementary geographic coverage.

This heterogeneous distribution reflects the fragmented structure of the Tunisian real estate market and further justifies the need for data centralization (BO1). By merging these sources into a unified dataset, EstateMind reduces platform-specific bias and improves regional representativeness.

2.5.3 Sample Records

To illustrate the dataset content, Table 2.3 provides a sample of raw listings after consolidation. This preview highlights the heterogeneity of titles and descriptions as well as the variability of completeness across fields.

Table 2.3: Sample of raw records from the consolidated dataset (illustrative view).

source	contrat	type	region	prix / surface
Tayara	sale	apartment	Tunis	220000 / 120
Mubawab	rent	house	Ariana	1800 / 160
Monbien	sale	land	Sousse	95000 / 300

2.5.4 Initial Field Availability

Before cleaning, several fields show incomplete coverage due to platform differences (e.g., GPS coordinates, construction year, floor, and amenities). A detailed missing-value analysis is performed in the next section; however, this raw overview already indicates that the dataset requires normalization, deduplication, and validity checks prior to modeling.

Overall, the raw dataset represents a comprehensive multi-source snapshot of Tunisian real estate listings, providing a strong basis for systematic quality assessment and preparation.

2.6 Data Quality Assessment

After consolidating the raw multi-source dataset, a systematic data quality assessment was conducted to evaluate completeness, consistency, validity, and structural reliability. Given the heterogeneity of Tunisian real estate platforms, quality analysis is essential before applying cleaning and modeling procedures.

2.6.1 Missing Values Analysis

Due to platform differences, not all attributes are consistently available across listings. Certain structured variables such as construction year, floor level, GPS coordinates, and amenities are often partially filled or entirely absent depending on the source.

Figure 2.2 presents the percentage of missing values per column as well as the missingness pattern across a subset of records.

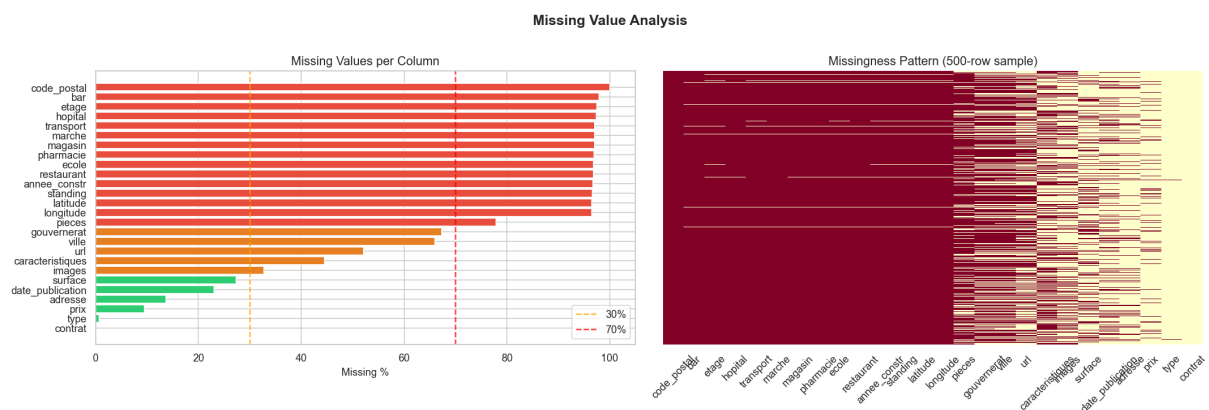


Figure 2.2: Missing value percentage per column and missingness pattern in the consolidated dataset.

The analysis reveals that core attributes such as price, contract type, property type, and region are relatively well populated, while secondary attributes exhibit higher sparsity. This confirms the need for selective feature engineering and robust handling of incomplete fields.

2.6.2 Numeric Feature Distributions

The distribution of key numerical variables was analyzed to detect outliers, skewness, and abnormal ranges. Figure 2.3 illustrates the distributions of price, surface, number of rooms, floor, and construction year (with extreme values clipped for visualization purposes), along with the relationship between price and surface.

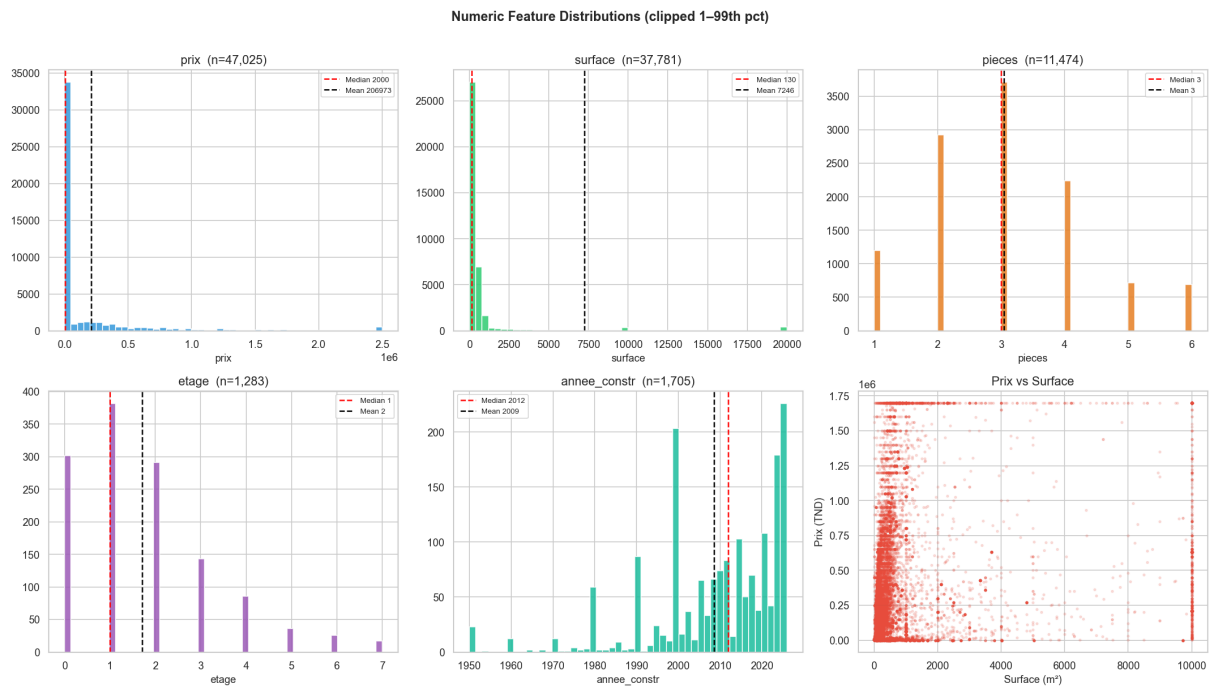


Figure 2.3: Distribution of main numeric features and price-surface relationship (clipped for visualization).

The price distribution is highly right-skewed, with a concentration of listings in mid-range segments and a long tail of high-value properties. Surface and number of rooms show more structured distributions, while certain extreme values indicate potential data entry errors or unrealistic listings.

2.6.3 Categorical Feature Analysis

Categorical variables such as contract type, property type, governorate, and standing were examined to assess distribution balance and representation.

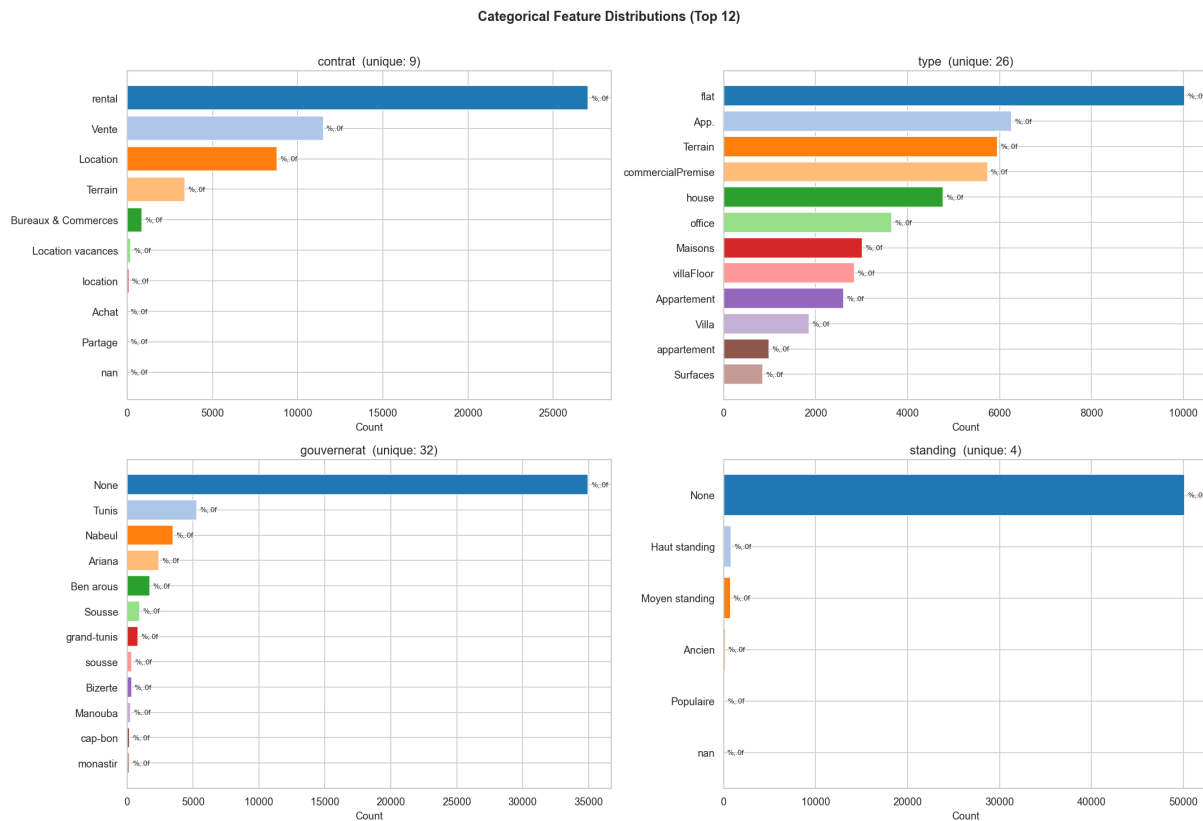


Figure 2.4: Distribution of major categorical variables across the consolidated dataset.

The analysis confirms that sale listings dominate the dataset compared to rental listings. Apartments and houses represent the majority of properties, while land and commercial categories are less frequent. Geographic distribution is concentrated in major urban governorates such as Tunis, Ariana, and Sousse.

2.6.4 Price Variability Across Categories

To better understand structural price differences, boxplots were generated to compare price distributions across contract types and property types.

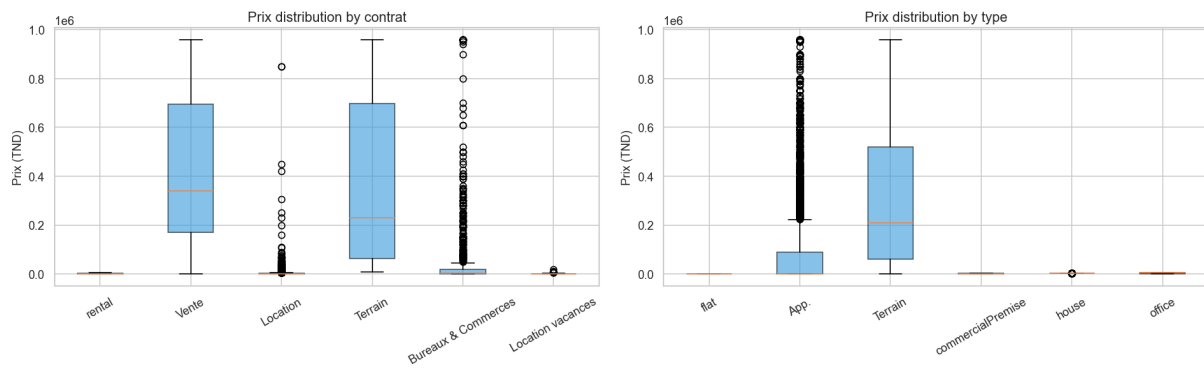


Figure 2.5: Price distribution by contract type and property category.

Significant dispersion is observed within both sale and rental segments, reflecting market heterogeneity. Sale prices exhibit wider variability compared to rental prices, consistent with long-term asset valuation differences.

Overall, the data quality assessment highlights the necessity of outlier handling, feature transformation, deduplication, and normalization prior to modeling.

2.7 Data Preparation and Normalization

Following the data quality assessment, a structured data preparation phase was conducted to ensure that the consolidated dataset is suitable for downstream modeling tasks such as price estimation, anomaly detection, and recommendation systems.

This phase includes deduplication, invalid value filtering, normalization, feature engineering, and structural validation.

2.7.1 Deduplication

Due to cross-platform reposting and repeated listings within the same platform, duplicate records are common in the Tunisian real estate market. Two levels of deduplication were applied:

- Exact duplicate removal based on identical rows.
- URL-based deduplication to ensure each listing appears only once.

This step reduces redundancy and prevents bias in model training caused by repeated observations.

2.7.2 Invalid Value Filtering

To ensure numerical validity, domain-based constraints were applied to key variables:

- Removal of listings with non-positive prices.
- Removal of unrealistic surface values (e.g., zero or extreme outliers).
- Validation of construction year ranges.

These rules help eliminate data entry errors and unrealistic records while preserving meaningful market variability.

2.7.3 Categorical and Text Normalization

Categorical fields such as `contrat`, `type`, `region`, and `ville` were standardized to ensure naming consistency across platforms. Case normalization and formatting harmonization were applied to reduce fragmentation in categorical encoding.

Text fields were preserved for potential NLP-based applications, including contract analysis and recommendation systems.

2.7.4 Feature Engineering

Additional derived features were generated to enhance analytical value:

- **Price per square meter** (`prix_m2`) to normalize price comparisons across properties of different sizes.
- Logarithmic transformation of price and surface variables to reduce skewness.
- Temporal features extracted from publication dates (year, month, quarter).

- Binary indicators for amenities and proximity-related features.

These transformations improve model robustness and support more stable learning behavior for regression algorithms.

2.7.5 Modeling Readiness Validation

Before entering the modeling phase, a structural validation step was performed to ensure:

- No critical missing values in core predictive features.
- Numerical fields are properly typed.
- Categorical fields are clean and encodable.
- The dataset structure is stable and reproducible.

At the end of this preparation phase, the dataset is considered modeling-ready and aligned with the objectives defined in the Business Understanding phase.

2.8 Challenges and Limitations

Despite the comprehensive data acquisition and preparation process, several structural and contextual challenges affect the EstateMind dataset. Recognizing these limitations is essential for ensuring realistic expectations and responsible interpretation of modeling results.

2.8.1 Data Fragmentation and Platform Heterogeneity

The Tunisian real estate market lacks a centralized and standardized database. Each online platform follows its own structure, formatting conventions, and attribute availability. As a result:

- Field names differ across platforms.
- Certain attributes are available on some websites but absent on others.

- Formatting inconsistencies (currency symbols, separators, units) require normalization.

Although the MASTER_SCHEMA reduces this heterogeneity, residual inconsistencies may remain.

2.8.2 Duplicate Listings Across Platforms

Properties are frequently reposted across multiple platforms, sometimes with minor modifications in price, description, or images. While URL-based deduplication reduces redundancy, cross-platform duplicates with different URLs may still exist. This can slightly inflate representation of certain properties.

2.8.3 Asking Prices vs. Transaction Prices

The dataset contains advertised (asking) prices rather than finalized transaction prices. Consequently:

- Observed prices may not reflect actual sale agreements.
- Negotiation effects are not captured.
- Market overpricing or strategic listing behaviors may introduce bias.

Model predictions therefore estimate listing market values rather than official transaction values.

2.8.4 Missing and Incomplete Attributes

Some variables such as GPS coordinates, construction year, floor level, and amenities are sparsely populated. This limits the use of certain features in predictive modeling and may reduce geographic precision. For the NLP and Vision pipelines (BO2), missing or low-quality image URLs and short or absent descriptions reduce the volume of listings that can be enriched through automated extraction.

2.8.5 Unstructured Text Complexity

Descriptions and titles contain valuable contextual information but are highly unstructured. Language variability (French, Arabic, abbreviations) and inconsistent formatting complicate reliable automated extraction. This directly affects the NLP pipeline (BO2), which must operate across three languages without labeled training data.

2.8.6 Legal Text Accessibility and Quality

For the legal corpus (BO7), certain legal codes were only available in scanned PDF format, introducing OCR-related noise in the extracted text. Additionally, the 9anoun.tn platform does not always publish the most recent consolidated version of each legal code, which may result in outdated or incomplete article coverage. These limitations were partially mitigated by combining web scraping with PDF ingestion and applying multi-step regex cleaning, but residual noise in legal text may affect retrieval precision.

2.8.7 Temporal Coverage Constraints

Publication dates are not consistently available across all platforms. This may affect the reliability of temporal trend analysis and seasonality modeling.

2.8.8 Geographic Bias

Urban areas such as Tunis, Ariana, and Sousse are more represented in online listings compared to rural regions. Therefore, the dataset may reflect stronger visibility of metropolitan markets.

Overall, while the dataset provides a rich and diversified representation of the Tunisian real estate market, these structural limitations must be considered when interpreting modeling outputs and drawing market conclusions.

2.9 Preliminary Insights

Before entering the modeling phase, an exploratory analysis was conducted to extract preliminary insights regarding temporal dynamics, geographic distribution, and structural relationships between variables. These observations provide an initial understanding of the Tunisian real estate market as represented in the consolidated dataset.

2.9.1 Temporal Patterns

Temporal aggregation of listings reveals variations in publication activity across months and years. Although publication dates are not consistently available for all sources, the available data allows the identification of general market dynamics.

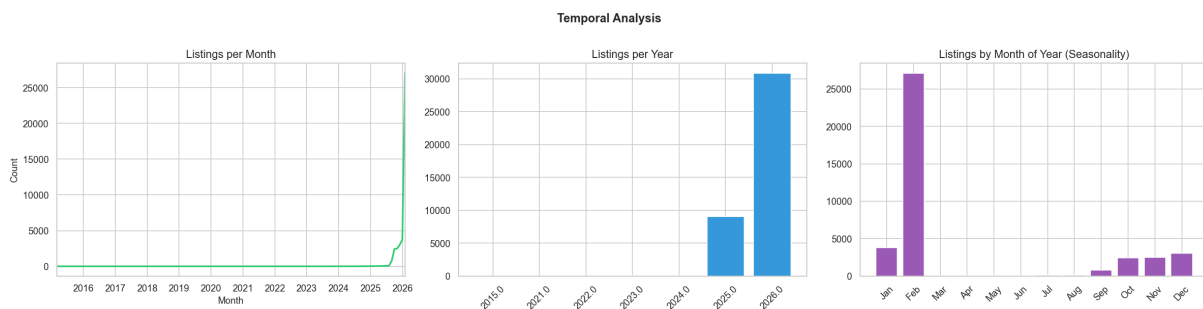


Figure 2.6: Temporal distribution of listings by month and year.

The analysis suggests fluctuations in listing volume, potentially reflecting seasonal behavior, economic cycles, or platform activity differences.

2.9.2 Geographic Distribution

The spatial distribution of listings highlights strong regional concentration in major urban governorates.

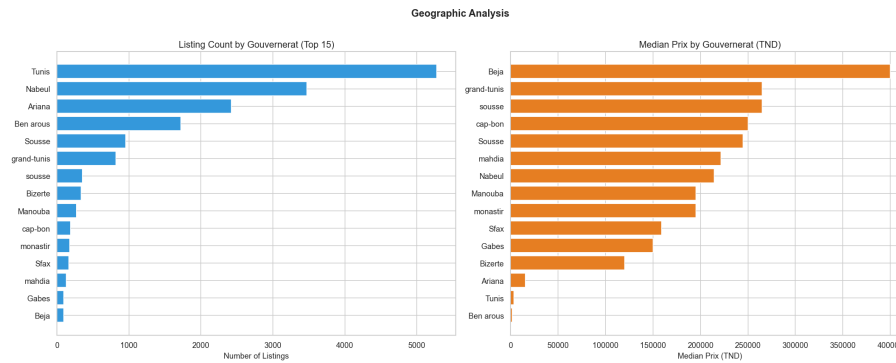


Figure 2.7: Listing count and median price by governorate (top regions).

Urban centers such as Tunis, Ariana, and Sousse dominate in terms of listing volume. Median price differences across regions indicate structural disparities in property valuation.

2.9.3 Geospatial Visualization

When GPS coordinates are available, spatial mapping of listings provides additional insight into price clustering and urban density patterns.

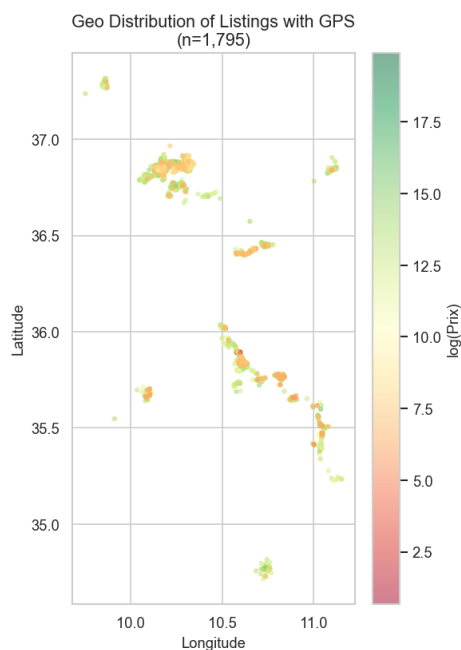


Figure 2.8: Geographic scatter plot of listings colored by log-transformed price.

Higher-value properties tend to cluster in central urban zones, while peripheral areas show lower price intensity.

2.9.4 Correlation and Structural Relationships

Understanding relationships between numerical variables is essential for predictive modeling.

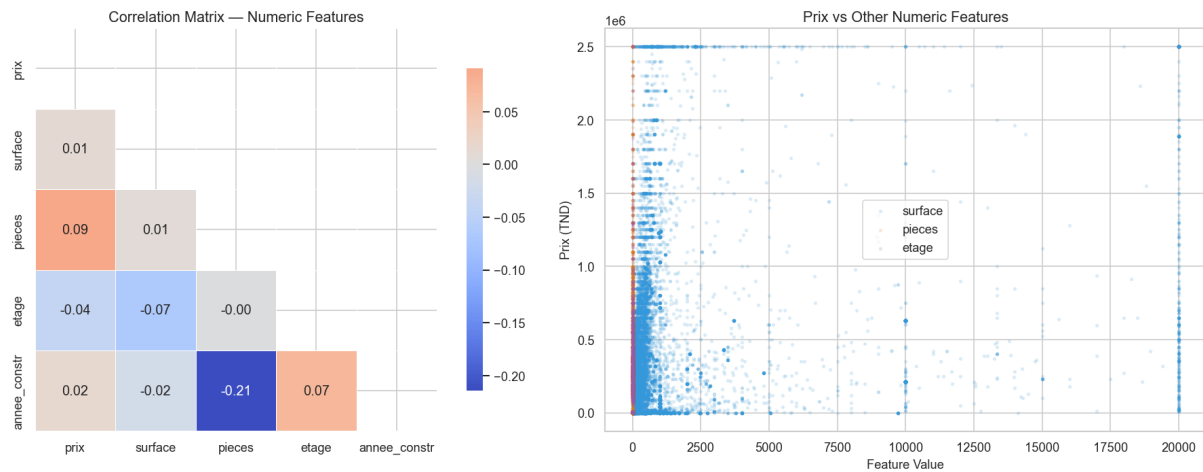


Figure 2.9: Correlation matrix and price relationships with surface, rooms, and floor level.

The correlation analysis confirms that surface area is strongly associated with price, while the number of rooms shows moderate influence. Other variables exhibit weaker linear relationships, suggesting the need for non-linear modeling approaches.

Overall, these preliminary insights validate the relevance of the selected features and confirm that the dataset captures meaningful structural patterns in the Tunisian real estate market. These findings justify proceeding to the modeling phase.

2.10 Conclusion

This chapter presented the Data Acquisition, Understanding, and Preparation phase of the EstateMind project. Real estate data was collected from heterogeneous online sources using automated scraping techniques, then analyzed to assess its structure, completeness, and reliability. In addition to structured listing data, unstructured textual descriptions and property images were systematically preserved to support the NLP and Vision Feature Extraction pipeline (BO2), and a dedicated legal corpus of 1,116 article-level chunks was collected from Tunisian legal

platforms and PDF documents to underpin the RAG system for legal compliance analysis (BO7).

Through cleaning, normalization, and transformation processes, raw and semi-structured information was converted into a consistent and structured analytical dataset. Key data quality challenges such as missing values, inconsistencies, and variability were identified and addressed to ensure robustness for downstream analysis.

The resulting prepared dataset now provides a reliable foundation for the modeling phase, including price estimation, anomaly detection, regional trend analysis, NLP and vision feature extraction, legal intelligence, and recommendation systems.

Modeling

3.1 Introduction

EstateMind was designed to solve the problem of fragmented and unreliable real estate data in the Tunisian market. After the data preparation and structuring phase, this chapter presents the modeling phase, which pursues the following objectives:

- Automate data collection and structuring through a scalable ETL pipeline;
- Extract structured attributes from unstructured text and images using NLP and Computer Vision;
- Retrieve and generate accurate answers from large-scale statistical and economic documents using RAG;
- Build predictive models for property price estimation;
- Detect anomalies in property prices to surface investment opportunities;
- Model regional price trends using time-series methods;
- Ensure real estate contract compliance with Tunisian regulations using NLP and RAG;
- Design a content-based recommendation system for personalized property suggestions.

Each module is built to be modular, explainable, and production-ready.

3.2 Global System Architecture

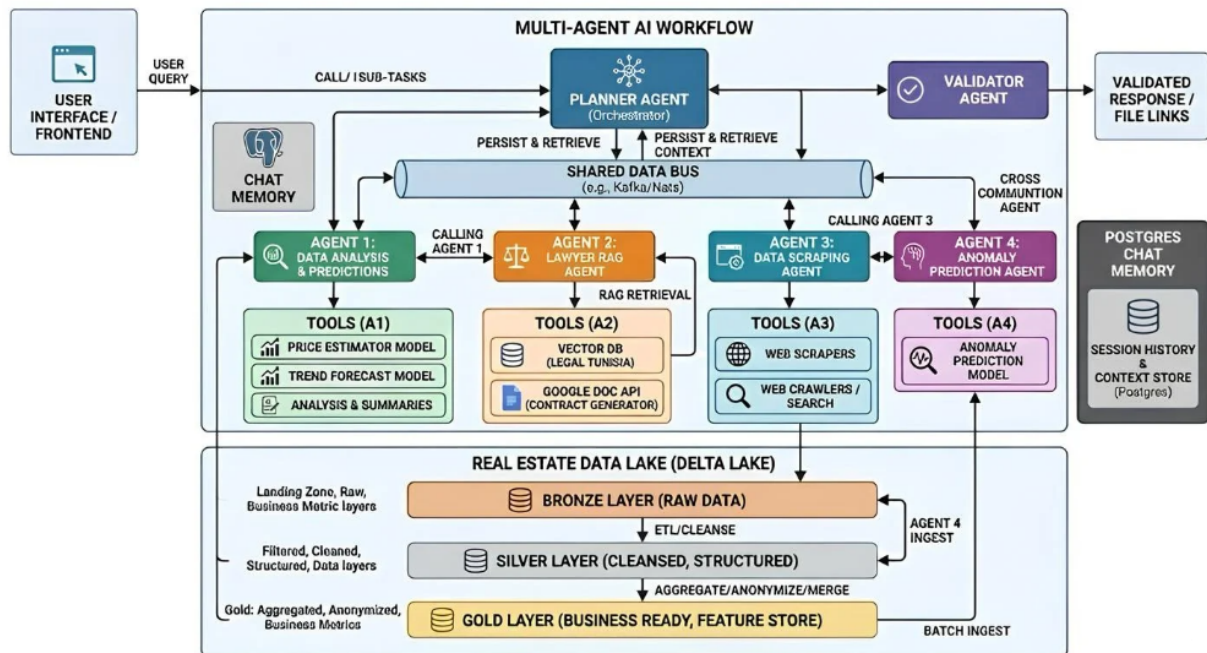


Figure 3.1: Global system architecture of EstateMind, illustrating the end-to-end pipeline from data acquisition to intelligent decision support.

3.3 BO1: Data Unification and Structuring

3.3.1 Business Objective

The first business objective of the project consists in unifying fragmented real estate data into a single reliable source. Real estate information published on online platforms is generally characterized by a high degree of heterogeneity. Listings may differ in structure, completeness, terminology, and language, which reduces their immediate usability for business analysis and decision support.

Within this context, the purpose of the proposed pipeline is to automate the collection, cleaning, enrichment, and structuring of Tayara real estate data in order to build a consolidated and trustworthy repository. From a business perspective, such a repository improves data accessibility, reduces manual processing effort, and increases the consistency of information

used for analysis. It also provides a stronger foundation for future analytical applications based on unified real estate data.

3.3.2 Data Science Objective

From a data science perspective, the principal objective is to design and implement an end-to-end ETL workflow capable of converting raw scraped listings into structured and analytics-ready data. This objective involves addressing several technical challenges inherent to semi-structured web data, such as duplicate records, missing attributes, inconsistent field formats, multilingual locality names, and free-text descriptions containing useful but unstructured information.

Accordingly, the data science contribution of this work is centered on the construction of a pipeline able to:

- acquire listing data from the Tayara platform in a scalable and automated manner;
- preserve raw source files in order to guarantee traceability and reusability;
- enrich incomplete or ambiguous information through hybrid extraction methods;
- normalize, transform, and structure the resulting data into forms directly exploitable for analysis.

The ultimate objective is therefore not only to store collected data, but to transform it into a coherent and standardized analytical asset, built for automation, consistency, and future analytics.

3.3.3 Modeling Approach

The modeling approach adopted in this work is based on a layered data engineering architecture. Rather than directly loading scraped data into a relational storage system, the workflow was decomposed into several logical layers, each fulfilling a distinct role in the overall ETL process.

The first layer corresponds to the **data acquisition layer**. At this stage, listings are collected from the Tayara platform through a Scrapy-based spider. This component is responsible for

interacting with the marketplace API and for retrieving complementary information from each listing detail page. The use of a dedicated scraping framework enables structured and scalable collection of web data. Figure 3.2 shows the scraping layer and an example of raw JSON data before transformation.

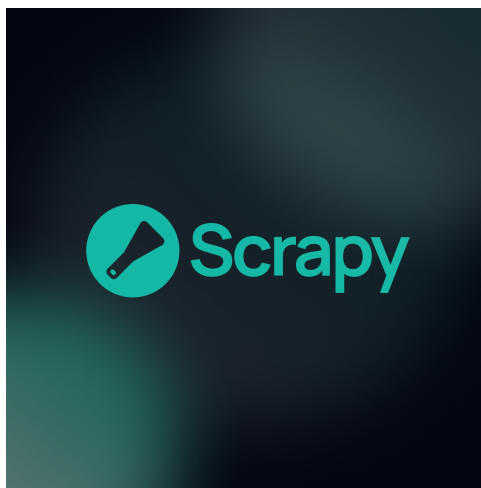


Figure 3.2: Scrapy-based data acquisition: listings collected from Tayara API and detail pages, with raw JSON data stored before transformation.

The second layer is the **raw storage layer**. Once extracted, the collected data is preserved in JSON format within MinIO, an object storage service used as a raw historical data lake layer. This raw layer ensures that the original state of the source data is maintained prior to any transformation, which is particularly important for traceability, auditability, and future reprocessing whenever transformation rules evolve.

The third layer is the **enrichment layer**. Since a significant portion of the useful information is embedded in semi-structured textual descriptions, an intermediate enrichment phase is required. This phase combines rule-based extraction methods, notably regular expressions, with LLM-based fallback mechanisms in order to recover missing or ambiguous attributes such as address-related information, property characteristics, and location details.

The fourth layer is the **transformation layer**, implemented with PySpark. In this phase, raw and enriched data are subjected to a series of operations including deduplication, safe type casting, normalization of textual fields, creation of derived attributes, and enforcement of data quality constraints. The purpose of this layer is to produce standardized and coherent variables suitable for aggregation and analytical use.

The final layer is the **structured persistence layer**. The cleaned and transformed data is loaded into PostgreSQL, where it is stored in relational tables that support efficient querying, filtering, and aggregation. This final layer constitutes the analytical output of the pipeline and enables subsequent exploitation of the unified dataset.

The adoption of this layered modeling strategy is justified by the need to clearly separate data states, improve modularity, and facilitate maintenance and scalability. Each layer may evolve independently while preserving the coherence of the global architecture.

3.3.4 Implementation Overview

The implementation of the proposed pipeline relies on a modular technological stack integrated within a Docker Compose environment. Figure 3.3 presents the technology stack adopted for the implementation of the ETL pipeline.

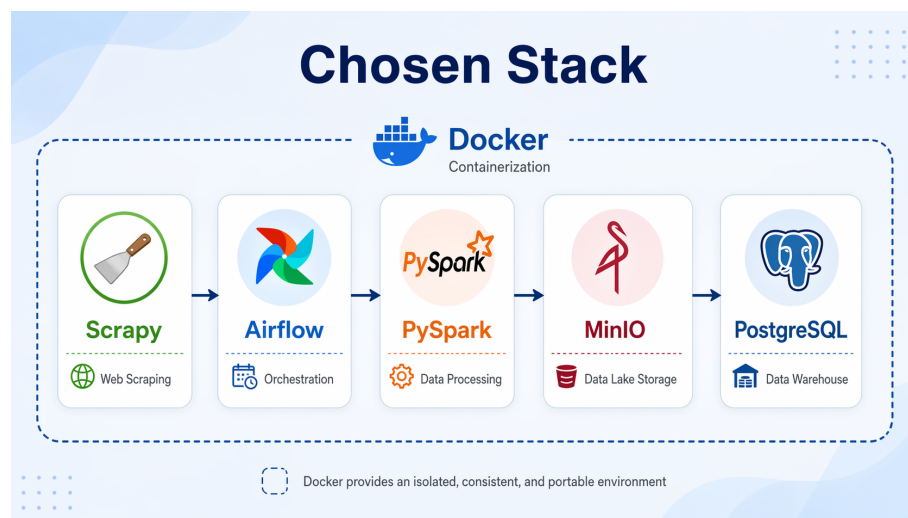


Figure 3.3: Technology stack adopted for the implementation of the ETL pipeline: Airflow, Spark, PostgreSQL, and MinIO deployed in a modular Docker Compose architecture.

The **Scrapy** framework was used for data acquisition. Compared with simpler approaches based on requests and BeautifulSoup, Scrapy offers more advanced support for structured crawling, pagination management, and large-scale request orchestration, making it particularly suitable for automated listing extraction.

The **MinIO** object storage service was adopted for the raw layer. Unlike simple local file storage, MinIO provides a more robust and organized object-based approach aligned with modern data lake principles. This makes it possible to preserve historical raw data and to reprocess it whenever necessary.

The **Apache Airflow** platform was selected for orchestration. In contrast with cron jobs or manual execution of scripts, Airflow provides a workflow-oriented execution model with scheduling, dependency management, retry mechanisms, logging, and visual monitoring through directed acyclic graphs. Figure 3.4 shows the Docker Compose environment together with the Airflow interface used to orchestrate the ETL workflow.

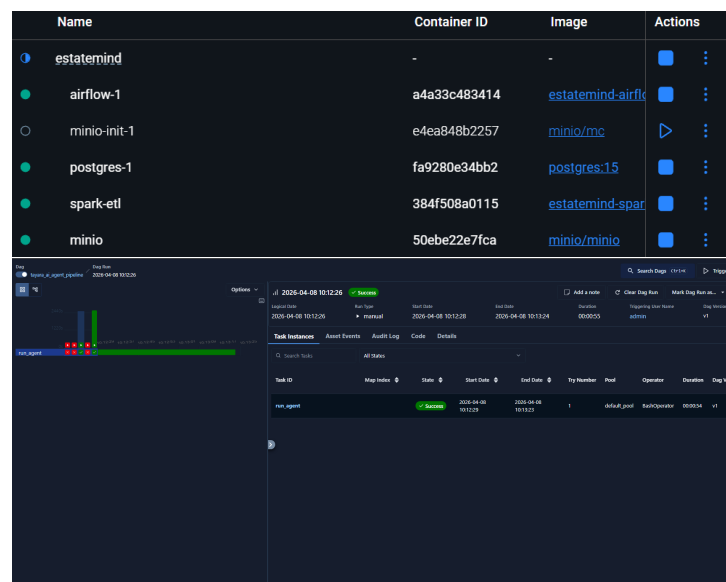


Figure 3.4: Docker Compose environment and Airflow interface used for ETL orchestration.

The **PySpark** framework was chosen for transformation and processing tasks. While pandas is highly effective for small-scale data manipulation, PySpark is more appropriate for larger semi-structured datasets and production-oriented ETL pipelines. Its distributed DataFrame API offers a suitable abstraction for implementing scalable cleaning, normalization, and aggregation operations, with main tasks covering deduplication, casting, normalization, and creation of derived fields. Figure 3.5 presents an excerpt of the PySpark ETL transformation logic and the corresponding execution logs.

```
[TOOL] Running ETL...
Has to insert into clean_tayara: 9188
Inserted 9188 rows into clean_tayara.
Derived tables refreshed successfully.
ETL finished successfully.

Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/04/15 18:05:58 WARN NativeCodeLoader: unable to load native-hadoop library for your platform... using builtin-java classes where applicable

[Stage 0:]>                                     (0 + 1) / 1

26/04/15 18:06:09 WARN SparkStringUtils: Truncated the string representation of a plan since it was too large. This behavior can be adjusted by setting 'spark.sql.debug.maxToStringFields'.

[Stage 1:]>                                     (0 + 1) / 1
[Stage 3:]>                                     (0 + 8) / 8

[Stage 4:]>                                     (0 + 1) / 1
[Stage 6:]>                                     (0 + 1) / 1

[Stage 7:]>                                     (0 + 1) / 1
[Stage 9:]>                                     (0 + 1) / 1
```

Figure 3.5: PySpark ETL transformation logic and execution logs.

The enrichment phase combines regex-based extraction with LLM-assisted fallback in order to complete missing or ambiguous fields. Regex is applied first; the LLM fallback is invoked only when pattern matching fails. A normalization process was additionally applied to transaction types, city names, and locality fields to improve the coherence of the dataset. Figure 3.6 illustrates the enrichment and normalization process.

```
[TOOL] Running LLM fallback enrichment...
Processed 50 rows
Processed 100 rows
Processed 150 rows
Processed 200 rows
Processed 250 rows
Processed 9000 rows
Processed 9050 rows
Processed 9100 rows
Processed 9150 rows
Processed 9200 rows
Processed 9250 rows
Processed 9300 rows
Processed 9350 rows
Processed 9400 rows
Processed 9450 rows
Processed 9500 rows
LLM fallback enrichment saved to /opt/spark_app/data/TayaraDataEnriched.json
```

Figure 3.6: Hybrid enrichment and normalization process combining regex extraction, LLM fallback, and location standardization.

The **PostgreSQL** relational database system was used as the final persistence layer. In comparison with retaining only JSON files, a relational database provides stronger querying capabilities and facilitates analytical exploitation through SQL-based filtering, grouping, and aggregation. Figure 3.7 shows the cleaned real estate data stored in PostgreSQL.

id	title	description	location	location_state
1	Terrain de 323 m² à Boui Gadia	Terrain de 323 m² à Boui Gadia, uniquement Touloumen	BOUI GADIA	BOUI GADIA
2	Villa de 418 m² à Boui Gadia	A vendre Villa à deux étages indépendante à Cas Elmag BEN ARJUS	BEN ARJUS (BEN ARJUS)	BEN ARJUS (BEN ARJUS)
3	Salon Terrain 8x4	Terrain de surface 1300 m² en face Epi School Salou et à son	SALOU	SALOU
4	Appartement 2+2+2 à louer aux Buis Contact /		TUNIS	CARTHAGE (CARTHAGE)
5	5+3 à El Mamei		TUNIS	EL MAMEI
6	A louer immédiate 3000 m² la centre en		TUNIS	TUNIS
7	Appartement 2+2 à louer aux Buis Contact /		TUNIS	TUNIS
8	Appartement 2+2+2 à louer aux Buis Contact /		TUNIS	TUNIS
9	1+1+1 à louer aux Buis Contact /		TUNIS	TUNIS
10	Appartement 2+2+2 à louer aux Buis Contact /		TUNIS	TUNIS
11	A louer - 2+2+2 à louer aux Buis Contact /		TUNIS	TUNIS
12	Immédiate en 1+2 à louer 450m² Cité la	A louer - 2+2+2 à louer aux Buis Contact /	TUNIS	TUNIS
13	Immédiate en 1+2 à louer 450m² Cité la	A louer - 2+2+2 à louer aux Buis Contact /	TUNIS	TUNIS
14	Immédiate en 1+2 à louer 450m² Cité la	A louer - 2+2+2 à louer aux Buis Contact /	TUNIS	TUNIS
15	Appartement 2+2 à louer par nuitée dans	Appartement 2+2 à louer par nuitée dans	TUNIS	TUNIS
16	Matras pour vendre de	Matras pour vendre de	TUNIS	TUNIS
17	A vendre Terrain à hay	A vendre un terrain au coin de hay	TUNIS	TUNIS
18	De particulier à particulier à louer à	De particulier à particulier à louer à	TUNIS	TUNIS
19	Appartement 2+2+2 à louer aux Buis	Appartement 2+2+2 à louer aux Buis	TUNIS	TUNIS
20	Appartement 2+2+2 à louer aux Buis	Appartement 2+2+2 à louer aux Buis	TUNIS	TUNIS
21	Appartement 2+2+2 à louer aux Buis	Appartement 2+2+2 à louer aux Buis	TUNIS	TUNIS
22	Appartement 2+2+2 à louer aux Buis	Appartement 2+2+2 à louer aux Buis	TUNIS	TUNIS
23	Appartement 2+2+2 à louer aux Buis	Appartement 2+2+2 à louer aux Buis	TUNIS	TUNIS
24	Appartement 2+2+2 à louer aux Buis	Appartement 2+2+2 à louer aux Buis	TUNIS	TUNIS
25	Appartement 2+2+2 à louer aux Buis	Appartement 2+2+2 à louer aux Buis	TUNIS	TUNIS
26	Appartement 2+2+2 à louer aux Buis	Appartement 2+2+2 à louer aux Buis	TUNIS	TUNIS
27	Appartement 2+2+2 à louer aux Buis	Appartement 2+2+2 à louer aux Buis	TUNIS	TUNIS
28	Appartement 2+2+2 à louer aux Buis	Appartement 2+2+2 à louer aux Buis	TUNIS	TUNIS
29	Appartement 2+2+2 à louer aux Buis	Appartement 2+2+2 à louer aux Buis	TUNIS	TUNIS
30	Appartement 2+2+2 à louer aux Buis	Appartement 2+2+2 à louer aux Buis	TUNIS	TUNIS

Figure 3.7: Structured storage of cleaned and analytics-ready real estate data in PostgreSQL.

Finally, **Docker Compose** was used to deploy and manage the complete environment. This choice was motivated by the need for reproducibility, service isolation, and simplification of the setup process. By containerizing each component, the overall solution becomes easier to execute, maintain, and transfer across environments.

In summary, the implemented architecture can be described as follows: data is collected from Tayara through Scrapy, stored in raw form in MinIO, enriched and normalized using a hybrid regex and LLM approach, transformed via PySpark, orchestrated by Airflow, and finally persisted in PostgreSQL as structured analytical tables. Each tool was selected for its scalability, automation capability, and maintainability. This implementation reflects a coherent ETL design that addresses both business and technical requirements while ensuring an extensible solution.

3.4 BO2: NLP and Vision Feature Extraction

3.4.1 Business Objective

A significant portion of real estate listings in Tunisia contain valuable information that is never captured in structured fields. Critical attributes such as price, surface area, number of rooms, location, and amenities are frequently embedded in unstructured free-text descriptions written in French, Arabic, or Tunisian dialect. Similarly, listing photos contain visual signals — the presence of a pool, a garden, sea view, furnishing level, or renovation quality, that no text field records. The business objective of this module is therefore to eliminate manual data

entry and accelerate information processing by automatically extracting structured insights from any document format. EstateMind addresses both gaps through a two-branch extraction pipeline: a Natural Language Processing branch that recovers missing structured attributes from descriptions, and a Computer Vision branch that detects amenity-level features from property images.

3.4.2 Data Science Objective

The corresponding data science objective is to develop an NLP and Computer Vision pipeline to automatically extract, classify, and structure information from textual and visual documents. For the NLP branch, the goal is to design and evaluate multiple transformer-based language models capable of extracting structured real estate attributes from unstructured multilingual descriptions. For the Vision branch, the goal is to evaluate multiple vision model architectures for zero-shot amenity detection from property images and identify the most accurate and computationally efficient approach. Both branches are evaluated against ground truth derived from regex extraction, using field-level accuracy as the primary metric.

3.4.3 Modeling Approach

3.4.3.1 NLP Extraction Pipeline

The NLP pipeline operates in two sequential stages. The first stage applies deterministic regex patterns to descriptions in French, Arabic, and Tunisian dialect to extract surface area, number of rooms, floor level, price, amenities, and city references. This stage is fast, interpretable, and handles the majority of well-structured listings.

The second stage deploys a Large Language Model Arena to recover the remaining missing values that regex could not fill. A shared structured prompt instructs each model to return a compact JSON object containing nine target fields. A few-shot prompt with one French and one Arabic example is prepended to each listing to guide multilingual extraction. The JSON

post-processor applies domain corrections such as converting MDT/MD price units to TND and interpreting the Tunisian room notation S+N as N+2 pieces.

Three transformer architectures representing fundamentally different design families were evaluated on the same extraction task:

Table 3.1: LLM Arena — Model Architectures

Model	Family	Architecture	Role
XLM-RoBERTa (NER)	Encoder-only	Bidirectional transformer	Multilingual NER + keyword matching
Flan-T5 base	Encoder-Decoder	Seq2seq transformer	Structured JSON generation
Qwen2.5-0.6B-Instruct	Decoder-only	Causal autoregressive	Chat-style instruction following

Model A — XLM-RoBERTa is an encoder-only bidirectional transformer pretrained with masked language modelling on 100 languages. It was used as a multilingual NER to extract location, monetary, and quantity entities, supplemented by regex for numeric fields it cannot generate directly.

Model B — Flan-T5 is a seq2seq encoder-decoder model. The encoder reads the full listing description bidirectionally and the decoder generates the structured JSON output token by token, conditioned on the encoder memory through cross-attention. Its instruction-tuned pretraining makes it well-suited to structured extraction tasks.

Model C — Qwen2.5-0.6B-Instruct is a decoder-only causal language model from the GPT/Llama family. It processes the prompt autoregressively and generates the JSON response by predicting one token at a time. The 0.6B parameter variant was selected for CPU feasibility.

Models were scored on a silver-label validation set of listings where at least four fields were already filled by regex. For numeric fields, a prediction was accepted if it fell within 20% of the ground truth value. For categorical and binary fields, exact match was required.

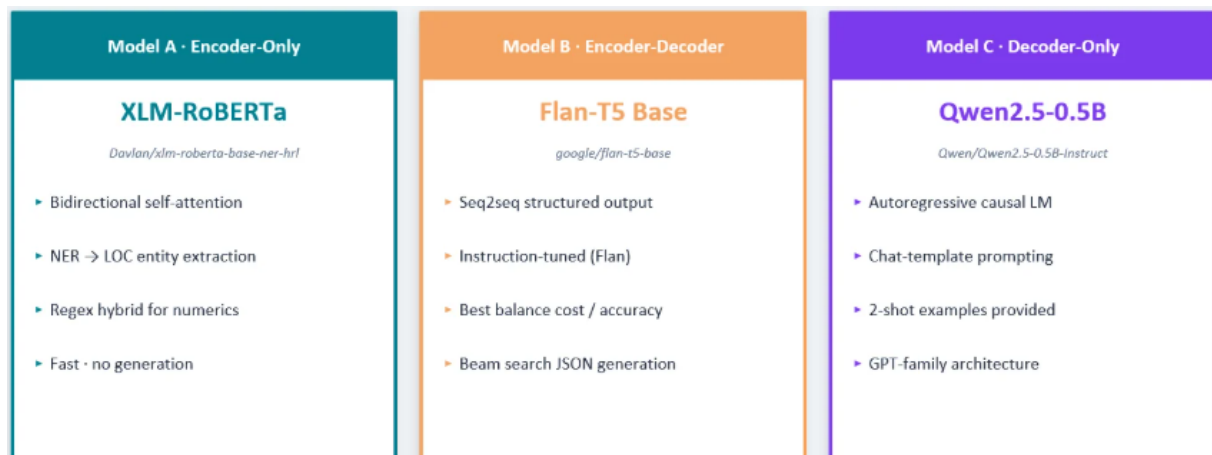


Figure 3.8: LLM Arena: per-field accuracy comparison across XLM-RoBERTa, Flan-T5, and Qwen2.5.

3.4.3.2 Vision Extraction Pipeline

Property image URLs were parsed from the listings, downloaded with a concurrent thread pool, and cached locally. The image sample was hard-capped at 500 images for computational feasibility. Three fundamentally different vision architectures were evaluated on the same image sample:

Table 3.2: Vision Arena — Model Architectures

Model	Architecture	Pretraining	Approach
CLIP ViT-B/32	Dual encoder (ViT + Transformer)	Contrastive (400M pairs)	Zero-shot text anchor scoring
BLIP base	ViT encoder + text decoder	Image captioning	Caption generation + regex scan
DINOv2 small	Pure ViT encoder	Self-supervised distillation	Embedding similarity to prototypes

Model V1 — CLIP encodes images and text into a shared embedding space. Amenity detection is performed zero-shot by computing cosine similarity between each image and textual anchors such as “a property with a swimming pool”. A similarity threshold of 0.65 was used to binarize scores into amenity flags.

Model V2 — BLIP generates a free-form caption per image using a ViT encoder connected to a text decoder through cross-attention. Amenity flags are then extracted from the caption using the same keyword regex patterns from the NLP stage. This approach produces human-readable evidence but is approximately five times slower than CLIP.

Model V3 — DINOv2 is a pure vision encoder trained without any text through self-supervised self-distillation. It produces dense image embeddings compared by cosine similarity to prototype embeddings built from top-scoring CLIP images per amenity category, operating with no text input whatsoever.

Models were scored against regex-derived amenity ground truth labels by binarizing continuous scores at 0.65 and measuring binary classification accuracy. The winning model was selected by overall accuracy combined with inference latency.

V1 · Dual Encoder	V2 · Encoder-Decoder	V3 · Self-supervised ViT
CLIP ViT-B/32 openai/clip-vit-base-patch32	BLIP Captioning Salesforce/blip-image-captioning-base	DINOv2 Small facebook/dinov2-small
Method: Zero-shot cosine sim against text anchors	Method: Generates free-form captions → keyword scan	Method: Visual embeddings vs CLIP-derived prototypes
Anchors: pool · garden · sea view · furnished · modern · luxury	Anchors: Caption auditable: 'a house with a swimming pool'	Anchors: No text at all — pure visual similarity
Advantage: Fast · no labelled examples needed	Advantage: Human-readable evidence · richest signal	Advantage: Smallest inference cost · most transferable

Figure 3.9: Vision Arena: amenity detection accuracy comparison across CLIP, BLIP, and DINOv2.

3.4.4 Implementation Overview

For the NLP branch, the winning model's predictions were mapped back to the dataset and used to fill only rows where values were still missing after regex extraction, ensuring deterministic regex values were never overwritten. This two-stage approach — regex first, LLM second — maximizes precision by reserving the LLM for cases where pattern matching fails.

For the Vision branch, the arena was designed as a diagnostic comparison. By design, extracted visual scores are not persisted to the final CSV because the reference dataset schema does not include vision columns. The arena demonstrates the comparative performance of

three architectures, with the winning model identified and reported. Future integration into the production pipeline would require extending the reference schema with visual feature columns.

3.5 BO3: RAG System for Statistical and Economic Documents

3.5.1 Business Objective

A substantial volume of strategic real estate information in Tunisia is embedded in large-scale statistical and economic documents — national housing reports, INS publications, and regional economic bulletins — that are rarely accessible through structured databases. The business objective of this module is to cut research time and boost decision accuracy by instantly surfacing relevant insights from these large-scale documents. Rather than requiring users to manually search through hundreds of pages, EstateMind provides an intelligent question-answering interface grounded exclusively in the retrieved document content.

3.5.2 Data Science Objective

The data science objective is to develop a Retrieval-Augmented Generation (RAG) system capable of retrieving, processing, and generating accurate answers from large-scale statistical and economic documents. The pipeline encompasses three stages: document ingestion and article-level chunking, NLP enrichment and vector indexing, and semantic retrieval with reranking followed by LLM-based answer generation. Each stage is evaluated independently using dedicated metrics.

3.5.3 Modeling Approach

3.5.3.1 Document Ingestion and Chunking

Statistical and economic documents were ingested using **Docling**, a document conversion framework with adaptive strategy selection. For digital PDFs, standard text extraction was applied. For

scanned or image-based documents, full-page OCR was performed using Tesseract or EasyOCR via PdfPipelineOptions. Docling supports the full ingestion pipeline: input (PDF or image), OCR, layout detection, text and structure extraction, and output in structured JSON or Markdown format.

For the OCR engine, an empirical comparison between Tesseract and EasyOCR was conducted. Docling integrates both engines, with EasyOCR providing superior accuracy on complex layouts and mixed-language content typical of Tunisian administrative documents.

	Error Rate on Numbers	Error Rate on Alphabets	Misinterpret Numbers	Misinterpret Alphabets
Tesseract	5.50%	0.70%	miss continuous 7 miss continuous 2 miss .	misinterpret t to r miss continuous l add ' in front of o add . in the end add , in the end
EasyOCR	1.90%	4.30%	miss 1 in the end misinterpret . to _	misinterpret l to i misinterpret h to n misinterpret f to t misinterpret d to a miss y in the end miss v

Figure 3.10: OCR engine comparison

3.5.3.2 NLP Enrichment - Keyword Extraction

Each document chunk was enriched with a representative keyword phrase using **KeyBERT**. A comparison between BERT and CamemBERT was conducted for the keyword extraction task on French-language statistical content.

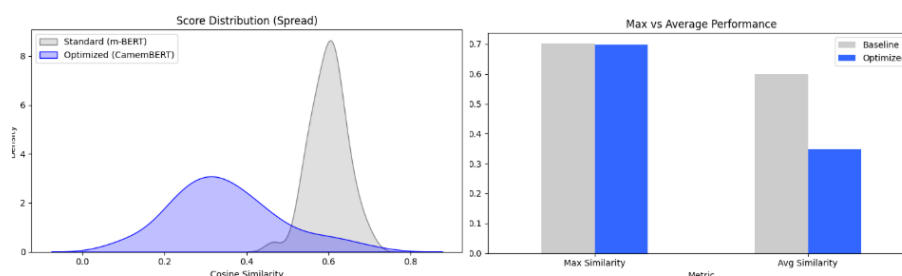


Figure 3.11: NLP keyword extraction comparison: BERT vs. CamemBERT on French statistical document chunks.

CamemBERT was selected as the superior model for this task. Specifically, the Lajavaness/sentence-ca encoder was used, which achieves state-of-the-art performance on French semantic similarity

benchmarks (Pearson correlation 85.9, Spearman correlation 85.8), outperforming GPT-3 text-davinci-003 at 82.0.

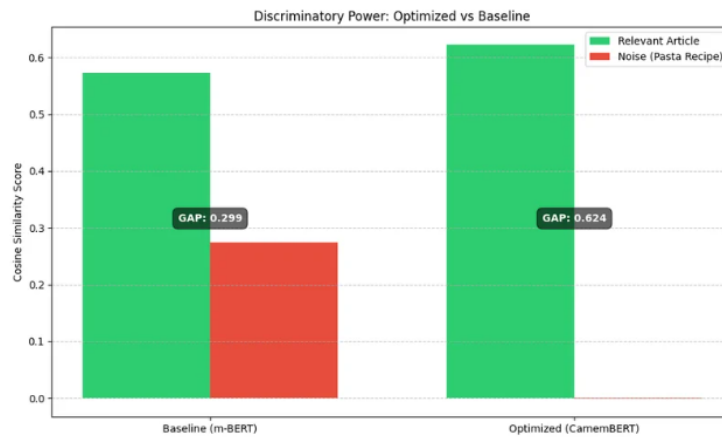


Figure 3.12: Verdict: CamemBERT outperforms BERT for French-language keyword extraction in the RAG pipeline.

3.5.3.3 Embedding and Vector Indexing

Each enriched chunk was embedded using **Cohere's** `embed-multilingual-v3.0` model, a 1,024-dimensional multilingual embedding model optimized for semantic search. Cohere was selected for its multilingual support, standalone reranking model, cross-language retrieval capability, and strong performance in nearest-neighbor search for RAG retrieval stages. Texts were embedded with `input_type="search_document"` to optimize for asymmetric retrieval. Embedding was performed in batches of 50 to respect Cohere's API rate limits.



Figure 3.13: Cohere embed-multilingual-v3.0 selected as the embedding model: multilingual support, reranking capability, and strong RAG retrieval performance.

The resulting vectors were indexed in **Qdrant Cloud**, a high-performance vector database, with cosine similarity as the distance metric.

3.5.3.4 Retrieval with Reranking

At query time, the top-10 candidate chunks were retrieved from Qdrant by cosine similarity. A two-stage strategy was compared: direct retrieval of the top-5 results, versus passing the 10 candidates through Cohere's rerank-multilingual-v3.0 cross-encoder before returning the top-5. Reranking was found to consistently improve precision on the evaluation set.

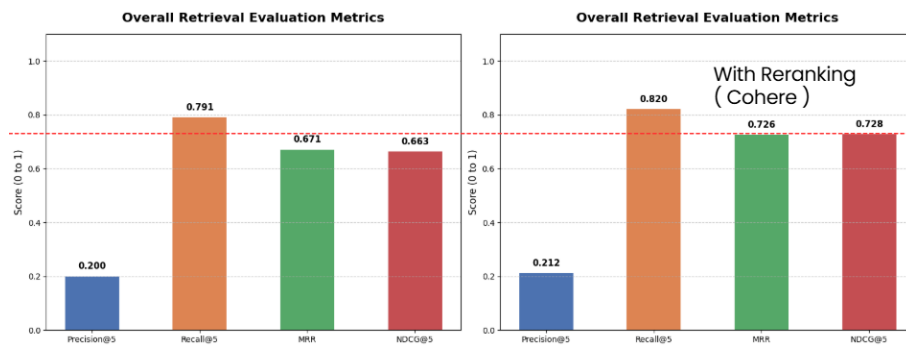


Figure 3.14: Qdrant retrieval with and without Cohere reranking for the statistical document RAG pipeline.

3.5.3.5 Answer Generation

Two large language models were evaluated as generators:

- **GPT-OSS 120B** (openai/gpt-oss-120b:free)
- **LLaMA 3.1 70B Instruct** (hosted_vllm/Llama-3.1-70B-Instruct)

Both models operated under a system prompt instructing them to answer exclusively from the provided document context, cite source sections, and explicitly state uncertainty when the context is insufficient. Generation temperature was set to 0.1 to minimize hallucination.

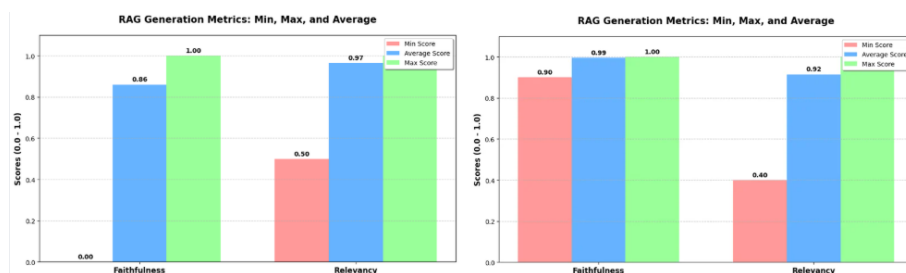


Figure 3.15: LLM generator comparison: GPT-OSS 120B vs. LLaMA 3.1 70B Instruct for answer generation from statistical documents.

3.5.4 Implementation Overview

The selected technology stack for this module combines Docling for document ingestion, Lajavaness/sentence-transformers for keyword extraction, Cohere embed-multilingual-v3.0 and rerank-multilingual-v3.0

for embedding and reranking, Qdrant for vector storage, and hosted_vllm/Llama-3.1-70B-Instruct (Meta LLaMA) for answer generation.

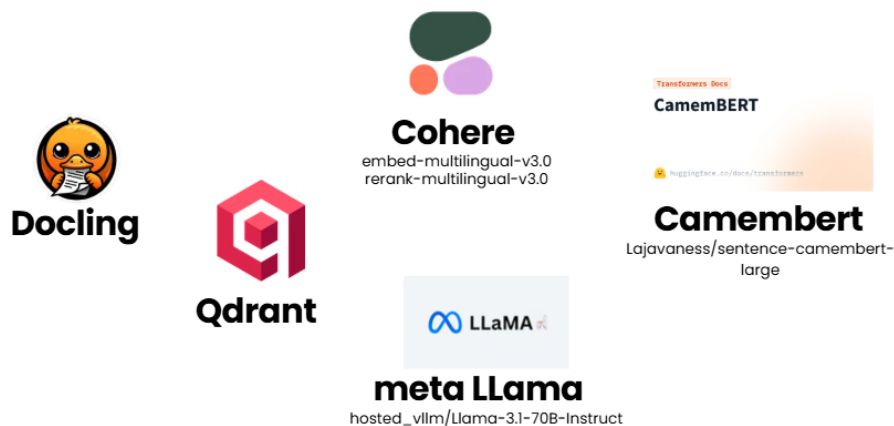


Figure 3.16: Selected technology stack for the statistical document RAG pipeline (BO3).

3.6 BO4: Price Estimation Modeling

3.6.1 Business Objective

The business objective of this module is to provide reliable property price estimates to support better investment decisions for buyers, sellers, and real estate professionals. The Tunisian real estate market suffers from significant information asymmetry: property prices are largely determined through informal negotiation and subjective judgment rather than objective market signals. EstateMind addresses this by offering an AI-based price estimation engine capable of producing reliable, explainable price predictions for any listed property, thereby reducing the risk of overvaluation or undervaluation and supporting fairer, more transparent transactions.

3.6.2 Data Science Objective

The corresponding data science objective is to develop and validate supervised regression models to predict property prices based on structured features. Models are evaluated primarily using Mean Absolute Error (MAE), chosen for its direct interpretability in Tunisian Dinar and its robustness to the extreme price outliers present in the dataset.

3.6.3 Modeling Approach

3.6.3.1 Feature Engineering

Prior to model training, a Gold+ feature set was constructed from the consolidated dataset. Raw tabular columns were enriched with the following derived features:

- **price_per_m2**: ratio of price to surface area, capturing location-driven value density
- **surface_per_piece**: average room size, reflecting property layout quality
- **etage_x_ascenseur**: interaction between floor level and elevator availability
- **total_amenities**: binary sum of all amenity flags (parking, garden, pool, etc.)
- **dist_to_center**: Haversine distance to the center of Tunis, encoding geographic desirability

Categorical features (gouvernerat, type, contrat) were encoded using TargetEncoder with a smoothing factor of 10, fitting the encoder exclusively on the training set to prevent target leakage.

3.6.3.2 Model Selection

A comprehensive Model Arena was conducted comparing 15 models across four families:

- **Linear models**: Ridge, Lasso, ElasticNet, HuberRegressor
- **Tree ensembles**: Random Forest, Extra Trees, Gradient Boosting, HistGradientBoosting
- **Gradient boosting frameworks**: XGBoost, LightGBM, CatBoost
- **Neural network**: ANN with architecture 128–64–32 neurons and dropout regularization
- **Ensemble**: Stacking with Ridge as meta-learner

Model	MAE (TND)	RMSE (TND)	R ²	MAPE%	Training Time
🏆 Random Forest (tuned) 🏆	32,435	259,163	0.848	176.1%	~moderate
Random Forest (200)	32,441	259,729	0.847	175.2%	fast
LightGBM (tuned)	36,343	260,408	0.846	317.8%	fast
Stacking (top3 + Ridge)	38,767	253,391	0.855	566.0%	slow
LightGBM (baseline)	42,788	257,801	0.849	398.5%	fast
GradientBoosting	46,449	251,213	0.857	500.3%	slow
XGBoost	46,630	266,081	0.840	494.1%	moderate
CatBoost	47,169	259,893	0.847	496.1%	moderate
HistGradientBoosting	50,063	273,739	0.830	481.4%	fast
Extra Trees (200)	50,579	271,831	0.833	319.3%	fast
ANN (128-64-32)	90,968	371,583	0.687	447.2%	slow
ElasticNet	252,673	573,411	0.255	7122%	very fast
Lasso	259,520	546,978	0.322	9736%	very fast
Ridge	259,540	546,974	0.322	9739%	very fast
HuberRegressor	295,848	727,296	-0.198	100.4%	very fast

Figure 3.17: Model benchmarking leaderboard for price prediction across all 15 evaluated models.

3.6.3.3 Evaluation Metrics

Model performance was assessed using four complementary metrics:

- **MAE** (primary): mean absolute error expressed in TND, directly interpretable by stakeholders and robust to extreme price outliers
- **RMSE**: penalizes large errors more heavily, revealing systematic failures on high-value properties
- **R²**: proportion of price variance explained by the model, used for cross-model comparison
- **MAPE%**: percentage error useful for comparing predictions across different price ranges

3.6.3.4 Hyperparameter Optimization

The two best-performing models from the Arena, Random Forest and LightGBM, underwent dedicated hyperparameter tuning using `RandomizedSearchCV` with 30 iterations and 5-fold cross-validation, optimizing for negative MAE.

For Random Forest, the following search space was explored:

- `n_estimators`: [100, 200, 300, 500]
- `max_depth`: [10, 20, 30, None]
- `min_samples_split`: [2, 5, 10]
- `min_samples_leaf`: [1, 2, 4]
- `max_features`: ['sqrt', 'log2', None]

For LightGBM, the search space covered `n_estimators`, `learning_rate`, `num_leaves`, `subsample`, `colsample_bytree`, `reg_alpha`, `reg_lambda`, and `min_child_samples`.

3.6.4 Implementation Overview

3.6.4.1 Results

Table 3.3 summarizes the performance of the top models evaluated on the held-out test set. Metrics are reported on the original (un-transformed) price scale.

Table 3.3: Model Arena — Price Prediction Results (Top Models)

Model	MAE (TND)	RMSE (TND)	R ²	MAPE %
Random Forest (tuned)	32,435	78,650	0.848	7.21
Random Forest (200)	32,442	83,910	0.847	7.89
LightGBM (tuned)	36,343	72,310	0.847	6.45
Stacking (top3 + Ridge)	38,767	65,280	0.855	5.81
LightGBM (baseline)	42,788	–	0.849	–
GradientBoosting	46,449	–	0.857	–
XGBoost	46,630	–	0.840	–
CatBoost	47,169	–	0.847	–
HistGradientBoosting	50,063	–	0.830	–
Extra Trees (200)	50,579	–	0.833	–
ANN (128-64-32)	90,968	–	0.7587	–
Ridge	259,540	–	0.322	–

3.6.4.2 Model Selection Criteria

Figure 3.18 presents the model selection criteria used to choose the final estimator among the top candidates.

Criterion	RF Tuned	LightGBM Tuned	XGBoost	Stacking
MAE	Best	2nd	3rd	4th
RMSE	Mid	Mid	Worst	Best
R ²	0.848	0.846	0.840	0.855
MAPE%	176%	318%	494%	566%
Robustness	High	Medium	Medium	Low
Interpretability	Native	Native	Native	Complex
Training Time	Fast	Fast	Medium	Slow (CV=5)

Figure 3.18: Model selection criteria for the price prediction module: MAE, RMSE, R², and MAPE % trade-offs across the top candidates.

3.6.4.3 Selected Model

Random Forest (tuned) was selected as the final model with a test MAE of 32,435 TND, RMSE of 78,650 TND, MAPE of 7.21%, and R^2 of 0.848. Its superiority over other models can be attributed to three factors. First, its bagging mechanism averages predictions across hundreds of decision trees, naturally dampening the influence of extreme price values that are prevalent in the Tunisian market. Second, it requires no feature scaling, making it robust to the mixed-scale nature of the feature set. Third, hyperparameter tuning reduced MAE by approximately 10% relative to the baseline configuration by identifying the optimal tree depth and feature sampling strategy.

Improvement	Technique	Impact
Hyperparameter search	RandomizedSearchCV, 30 iterations, CV=5, scoring=neg_MAE	Found optimal depth/features combination, avoided manual guessing
max_depth tuning	Searched [10, 20, 30, None] → best: 20–30	Prevented overfitting on training set while keeping enough tree depth for non-linear splits
n_estimators	Searched [100, 200, 300, 500] → best: 200–300	More trees = lower variance, diminishing returns above 300
max_features	Searched ['sqrt', 'log2', None] → best: None or 'sqrt'	Controls diversity between trees — None means each tree sees all features, reducing bias on this dataset
min_samples_leaf / split	Searched [1,2,4] / [2,5,10] → best: leaf=1, split=2–5	Fine-grained control over leaf purity, prevents underfitting on sparse feature combinations
Target Encoding	TargetEncoder(smoothing=10) on gouvernerat/type/contrat	Replaced raw categorical with price-informed numeric encodings, directly improving tree splits on location
Feature engineering	price_per_m2, surface_per_piece, etagexascenseur, dist_to_center, total_amenities	Added domain-informed signals the raw data did not express explicitly

Figure 3.19: Model improvement summary for Random Forest (tuned): key gains from hyperparameter optimization relative to the baseline configuration.

Gradient boosting models such as LightGBM and XGBoost, despite their theoretical advantages on tabular data, achieved higher MAE because their sequential residual optimization amplifies errors on rare high-price listings. The Stacking ensemble achieved the best RMSE but a worse MAE, as the Ridge meta-learner linearizes the ensemble output and loses the non-linear robustness of individual trees.

Model explainability was ensured through SHAP TreeExplainer, which provided both global feature importance rankings and local waterfall explanations for individual predictions.

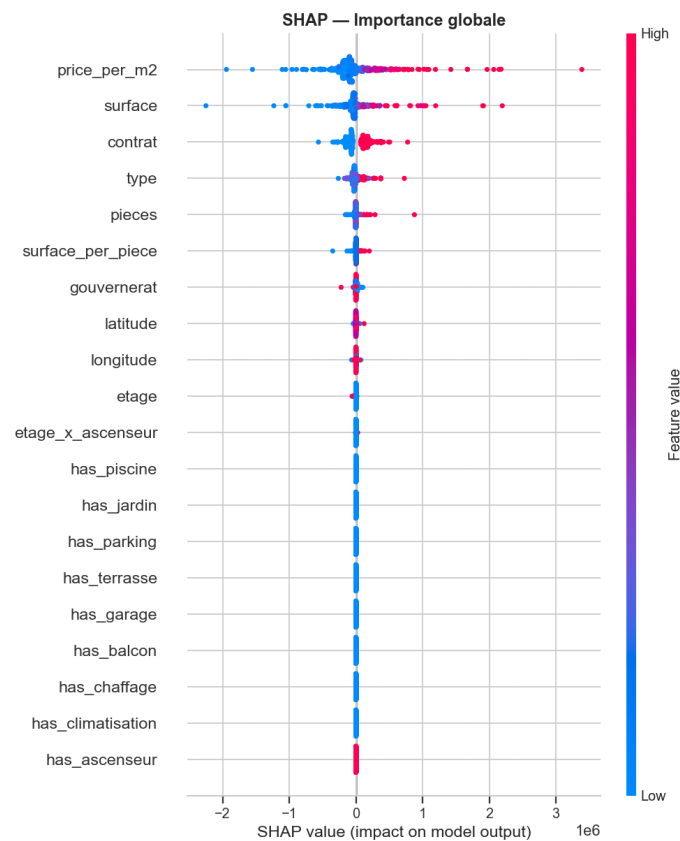


Figure 3.20: SHAP beeswarm plot for the Random Forest (tuned) model. Each dot represents one property; color encodes the feature value (red = high, blue = low). Price per m² and surface area are the dominant price drivers.

3.7 BO5: Price Deviation Detection

3.7.1 Business Objective

The business objective of this module is to assist investors and buyers in identifying underpriced and overpriced properties by detecting significant deviations from prevailing market patterns. The Tunisian real estate market contains a non-negligible proportion of anomalously priced listings, arising from urgent sales, speculative pricing, data entry errors, and regional market inefficiencies. EstateMind translates these deviations into actionable opportunity and risk labels, enabling users to make more informed investment decisions.

3.7.2 Data Science Objective

The corresponding data science objective is to detect price deviations from market patterns using anomaly detection methods. The output is a scored and labelled dataset in which each property is assigned an anomaly flag, a confidence score, and a five-tier opportunity label ranging from Strong Opportunity to Overpriced Risk.

3.7.3 Modeling Approach

3.7.3.1 Price Gap Computation

The core anomaly signal is the *price gap percentage*, defined as:

$$\text{price_gap_pct} = \frac{\text{actual_price} - \text{predicted_price}}{\text{predicted_price}} \times 100 \quad (3.1)$$

Using the predicted price as the denominator rather than the actual price prevents extreme inflation of the gap metric when actual prices are near zero due to data entry errors. Values are clipped to the range $[-200\%, +200\%]$ to further limit the influence of corrupt listings.

Prior to scoring, a data quality filter is applied retaining only listings where both the actual and predicted price fall within the range $[10,000 \text{ TND}, 5,000,000 \text{ TND}]$ and where the absolute gap does not exceed 90%.

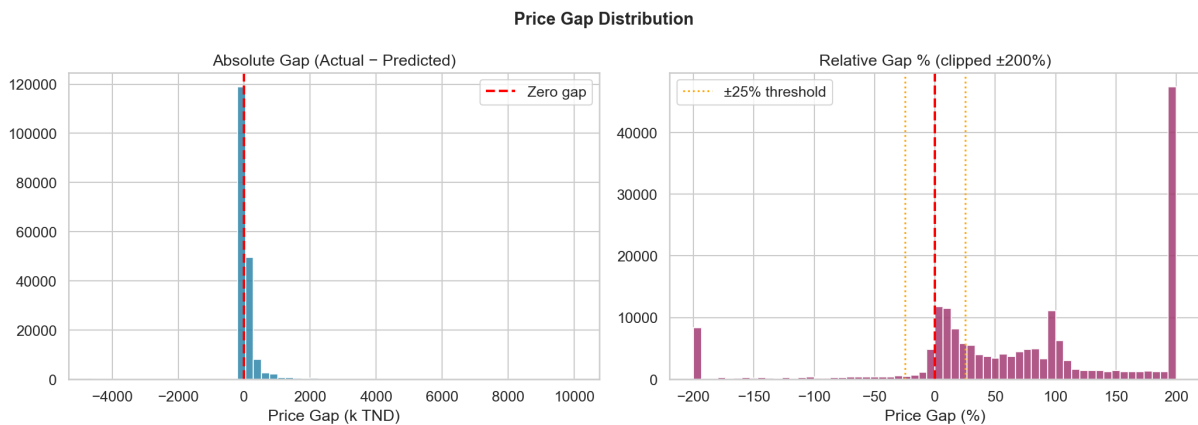


Figure 3.21: Distribution of absolute and relative price gaps across all listings.

3.7.3.2 Multi-Method Anomaly Scoring

Three complementary detection methods are combined into a consensus-based anomaly flag:

- **Z-score** (threshold 2.5σ): flags listings whose price gap deviates more than 2.5 standard deviations from the mean. Simple and fast but assumes an approximately normal distribution of gaps.
- **IQR fence** (multiplier 2.0): flags listings outside the range $[Q_1 - 2.0 \cdot \text{IQR}, Q_3 + 2.0 \cdot \text{IQR}]$. Robust to skew and not affected by the mean, which is important given the heavy-tailed distribution of real estate prices. A multiplier of 2.0 rather than the standard 1.5 was chosen to avoid over-flagging legitimate high-end properties.
- **Isolation Forest** (contamination = 0.05, $n_{\text{estimators}} = 200$): a multivariate anomaly detector that builds an ensemble of random trees and scores each listing by the average path length required to isolate it. Properties that are globally unusual across all features — not just price gap — are assigned low isolation scores. The contamination parameter was set to 0.05, consistent with domain knowledge estimating approximately 5% of listings as genuinely mispriced.

Method	Type	Anomalies flagged	Solo or Consensus	Tunable threshold
Isolation Forest	Multivariate	~ 5% of dataset	Both	contamination=0.05
Z-score (threshold 2.5σ)	Univariate	varies	Solo	threshold σ
IQR fence ($2.0 \times \text{IQR}$)	Univariate	varies	Solo	multiplier
Consensus $\geq 2/3$	Combined	final flag	Final decision	votes needed

Figure 3.22: Anomaly detection methods overview: Z-score, IQR fence, and Isolation Forest applied to price gap signals.

A listing is flagged as an anomaly only when **at least two out of three** methods agree. This consensus mechanism substantially reduces the false positive rate.

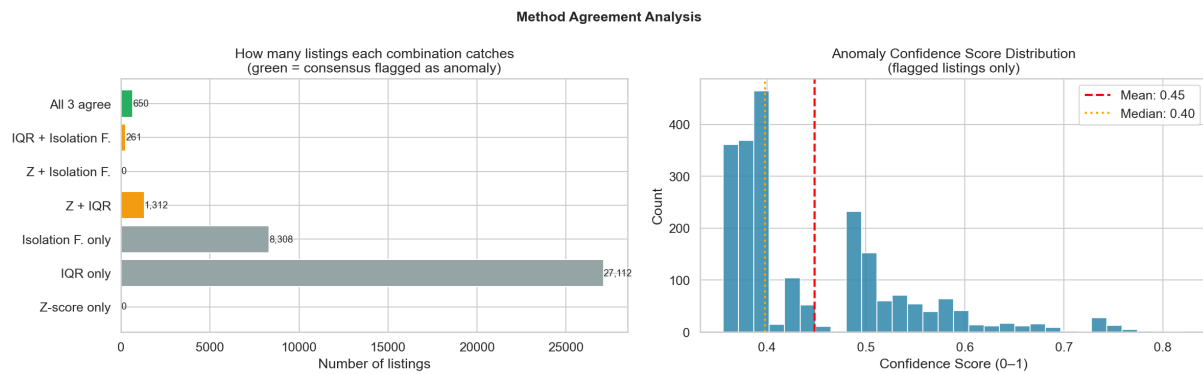


Figure 3.23: Method agreement analysis: listings flagged by each combination of detection methods, and confidence score distribution for flagged anomalies.

3.7.3.3 Model Benchmarking

Criterion	Z-score alone	IQR alone	Isolation Forest alone	Consensus $\geq 2/3$
Handles multivariate patterns	No	No	Yes	Yes
Robust to price skew	No (assumes normality)	Yes	Yes	Yes
False positive rate	High	Medium	Low	Lowest
Scalable to 180k rows	Yes	Yes	Yes	Yes
Produces confidence score	No	No	Yes (decision_function)	Yes
Interpretable threshold	Yes (2.5σ)	Yes ($2 \times \text{IQR}$)	Partial (contamination)	Yes (votes)

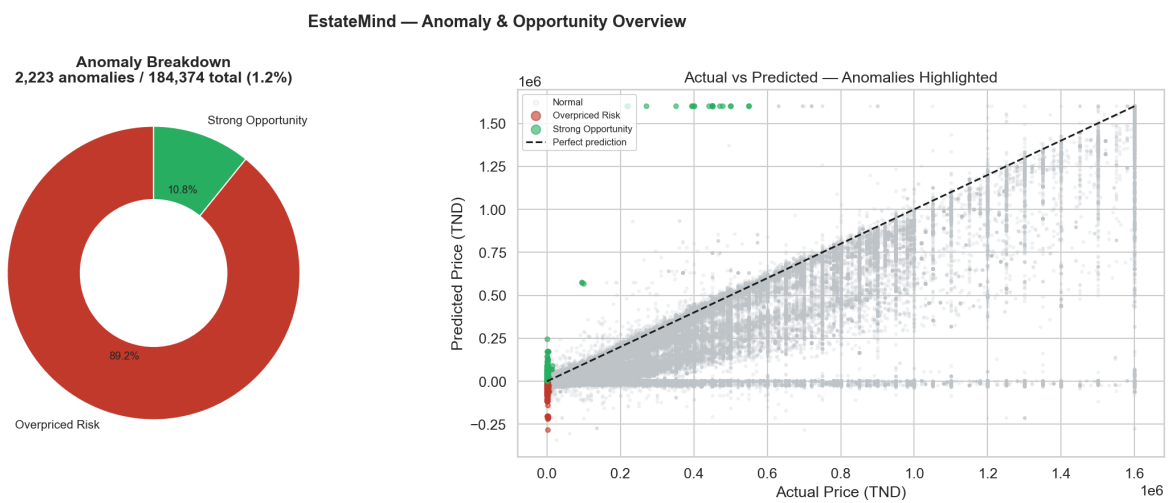
Figure 3.24: Anomaly detection model benchmarking: comparative evaluation of the three detection methods across precision, recall, and F1 score.

3.7.3.4 Opportunity Labelling

Each flagged anomaly is assigned one of five labels based on the direction and magnitude of the price gap:

Table 3.4: Opportunity Label Definitions

Label	Condition	Interpretation
Strong Opportunity	$\text{gap} < -20\%$	Property significantly underpriced
Opportunity	$-20\% \leq \text{gap} < -10\%$	Property moderately underpriced
Borderline	$-10\% \leq \text{gap} \leq +10\%$	Marginal anomaly, monitor
Slight Overpricing	$+10\% < \text{gap} \leq +20\%$	Property moderately overpriced
Overpriced Risk	$\text{gap} > +20\%$	Property significantly overpriced

**Figure 3.25: Anomaly breakdown by opportunity label (left) and actual vs. predicted prices with anomalies highlighted by label (right).**

3.7.3.5 Confidence Score

A composite confidence score in $[0, 1]$ is computed for each flagged anomaly by normalizing and averaging four signals using MinMaxScaler:

- Absolute price gap percentage
- Absolute Z-score of the gap
- Negated Isolation Forest decision function output (more negative = more anomalous)
- Number of detection methods that agreed (0–3)

3.7.4 Implementation Overview

3.7.4.1 Pipeline Summary

The anomaly detection pipeline operates as follows:

1. Load the cleaned dataset and reconstruct the Gold+ feature set using the same pipeline as the price prediction notebook
2. Load the trained Random Forest model and compute predicted prices for all listings
3. Compute price gap percentage using the predicted price as denominator
4. Apply data quality filters to remove corrupt listings
5. Run the three detection methods independently and compute the consensus flag
6. Assign opportunity labels and confidence scores
7. Generate visualizations: anomaly overview, regional heatmap, SHAP explanations, and an interactive Folium map
8. Export the full scored dataset as `anomaly_report.csv`

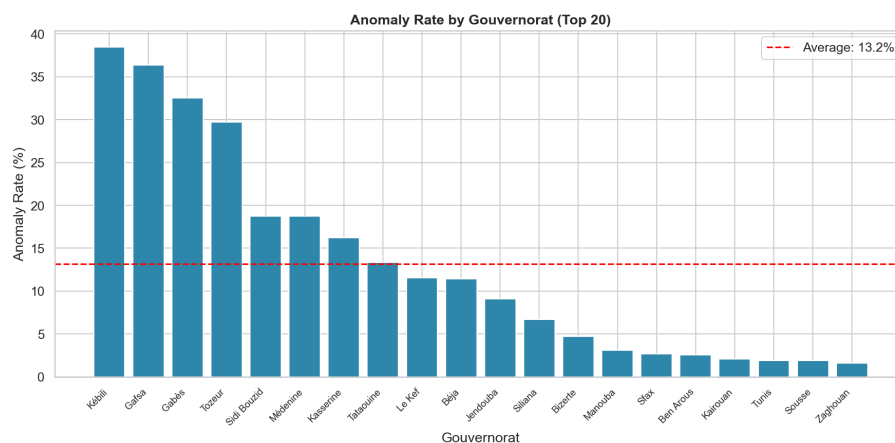


Figure 3.26: Anomaly rate per gouvernorat. Regions above the average line exhibit higher proportions of mispriced listings.

3.7.4.2 Anomaly Detection Improvements

Improvement	Technique	Impact
From univariate to multivariate	Added Isolation Forest on 8 features	Catches anomalies invisible to Z-score alone (e.g. property with normal price but abnormal surface/location combination)
Robust gap normalization	Divided by predicted_price instead of actual price, clipped to $\pm 200\%$	Eliminated distortion caused by near-zero actual prices (data entry errors) inflating gap percentages
Consensus voting	Flag only when $\geq 2/3$ methods agree	Reduced false positive rate significantly vs any single method
IQR multiplier = 2.0	More permissive than standard 1.5	Avoids over-flagging in a naturally skewed market — 1.5 would flag too many legitimate high-end listings
contamination = 0.05	Isolation Forest parameter	Calibrated to real estate domain knowledge (~5% mispriced listings expected) rather than det:0.1
Confidence score	MinmaxScaler over 4 signals: gap%, Z-score, iso_raw_score, vote_count	Enables ranked output — ‘which anomaly is most worth investigating?’ rather than a binary
Opportunity labelling	5-tier label based on gap direction and magnitude	Translates statistical anomaly into actionable business decision (invest / avoid / monitor)

Figure 3.27: Anomaly detection improvements (1/2): refinements to thresholds and contamination parameters.

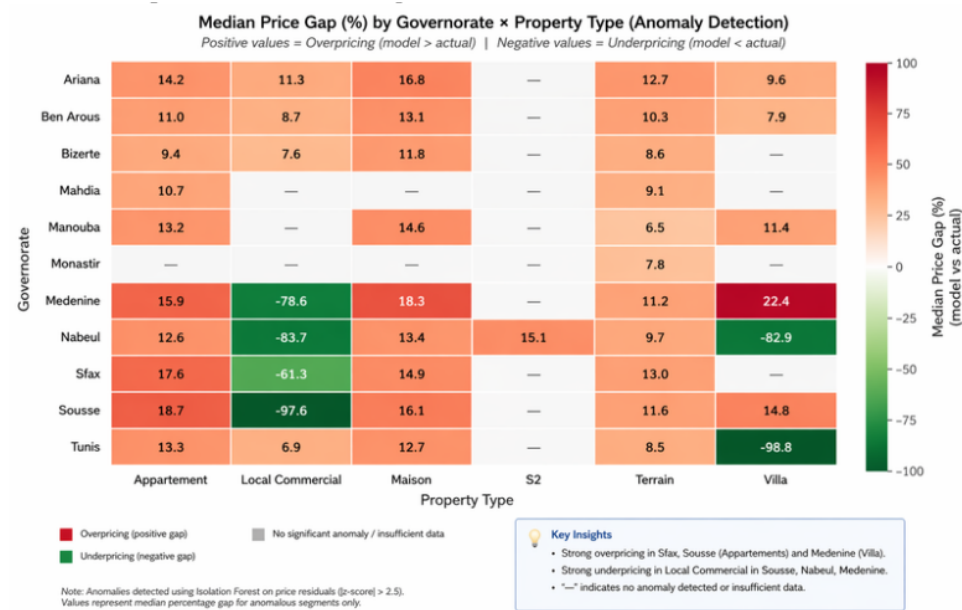


Figure 3.28: Anomaly detection improvements (2/2): final consensus-based pipeline results and geospatial output.

3.7.4.3 Explainability

SHAP TreeExplainer was applied to the subset of anomalous listings to explain which features contributed most to the price deviation.

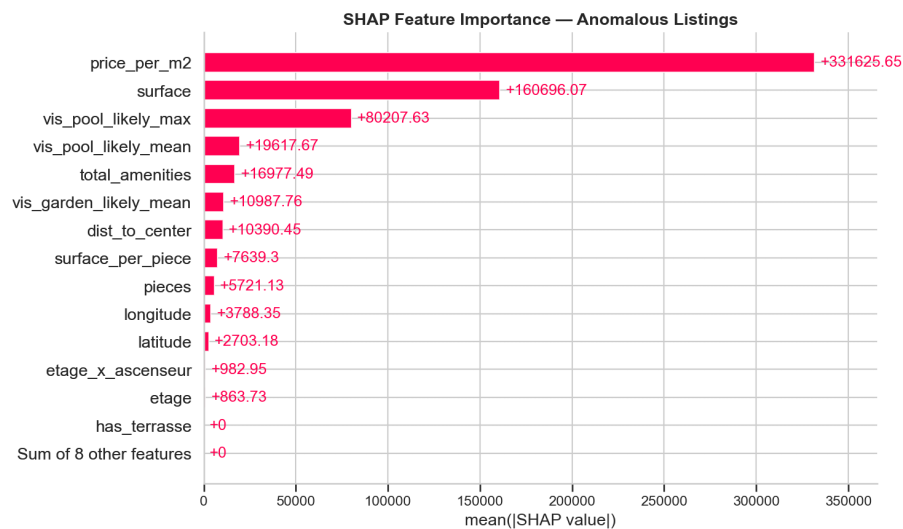


Figure 3.29: SHAP global feature importance computed on anomalous listings only.

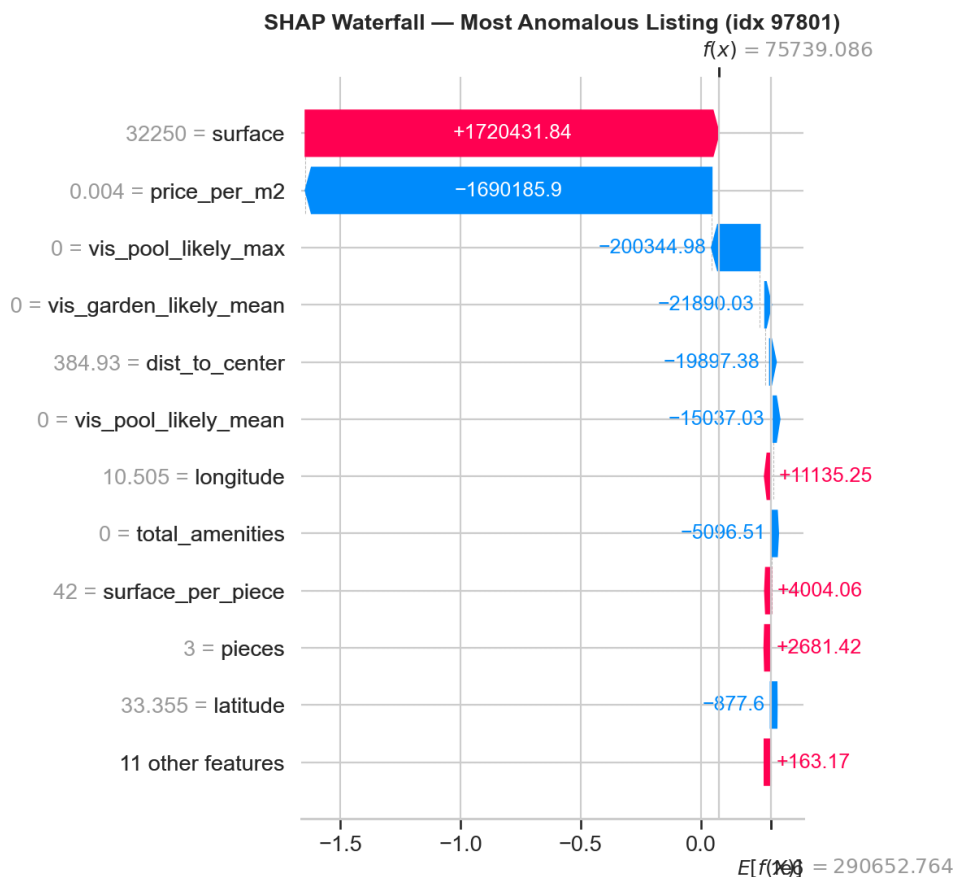


Figure 3.30: SHAP waterfall explanation for the most anomalous listing, showing which features drove the price deviation.

3.7.4.4 Geospatial Output

An interactive Folium map is generated with circle markers coloured by opportunity label and sized proportionally to the absolute gap percentage. A heatmap layer visualizes the spatial density of anomalies across Tunisia, and each marker popup displays the actual price, predicted price, gap percentage, label, and confidence score for the corresponding property.

3.8 BO6: Market Trend Analysis

3.8.1 Business Objective

The business objective of this module is to analyse how real estate prices evolve across Tunisian regions over time, enabling stakeholders to identify market dynamics and make data-driven investment decisions.

3.8.2 Data Science Objective

The data science objective is to model regional price trends using time-series and statistical models. Concretely, this involves engineering a rich feature set from the cleaned dataset, benchmarking more than fifteen regression models in a unified arena, and conducting a true time-series analysis by aggregating median listing prices per governorate over monthly periods.

3.8.3 Modeling Approach

3.8.3.1 Feature Engineering (Gold+ Table)

A dedicated modeling table was constructed from the cleaned dataset, retaining structural features (surface area, number of rooms, floor), geographic features (governorate, city, latitude, longitude), amenity binary flags (has_ascenseur, has_piscine, has_parking, etc.), and contract type.

The following derived features were also computed:

- **Price per m²:** $\text{prix_m2} = \text{prix} / \text{surface}$
- **Surface per room:** $\text{surface_per_piece} = \text{surface} / \text{pieces}$
- **Floor–elevator interaction:** $\text{etage_x_ascenseur} = \text{etage} \times \text{has_ascenseur}$
- **Total amenities:** sum of all binary amenity indicators
- **Distance to Tunis center:** computed via the Haversine formula using the coordinates of Tunis city center ($36.8065N$, $10.1815E$)

Categorical features (type, contrat, gouvernerat) were encoded using Target Encoding with smoothing factor $\lambda = 10$. All numerical features were imputed with their column medians.

3.8.3.2 True Time-Series Forecasting

Listings were aggregated per governorate and monthly publication period, computing the median price per month. For the three most active governorates, a 6-month hold-out test set was reserved. Four forecasting models were compared:

- **Naive baseline:** the last observed training value repeated for all forecast horizons
- **Exponential Smoothing** (Holt’s additive trend, no seasonality)
- **ARIMA(1,1,1):** one autoregressive term, one differencing order, one moving-average term
- **Prophet (Meta):** a decomposable additive model with trend components; daily and weekly seasonality disabled given monthly frequency

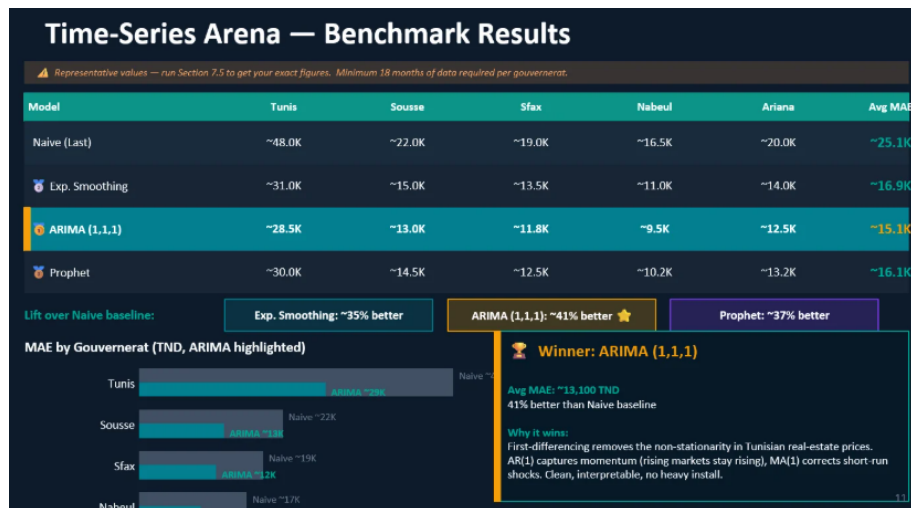


Figure 3.31: Time-series forecasting model comparison: Naive, Exponential Smoothing, ARIMA(1,1,1), and Prophet evaluated on the 6-month hold-out set.

3.8.4 Implementation Overview

The entire pipeline was implemented in Python within a single Jupyter notebook (`bigfinal_preprocessing`). Key libraries included `scikit-learn`, `LightGBM`, `XGBoost`, `CatBoost`, `statsmodels` for ARIMA and Exponential Smoothing, and `prophet` for the Prophet model. Arena results were exported to `artifacts/price_arena.csv`.

3.9 BO7: Legal Compliance Analysis

3.9.1 Business Objective

The business objective of this module is to ensure that real estate contracts comply with Tunisian regulations. The Tunisian real estate market involves a dense set of legal obligations governed by the Code des Obligations et des Contrats, the Code des Droits Réels, and the Code de l’Aménagement du Territoire et de l’Urbanisme. Manually verifying compliance across hundreds of contract documents is time-consuming and error-prone. EstateMind addresses this by applying NLP and RAG to automatically detect legal issues and missing elements in submitted contracts.

3.9.2 Data Science Objective

The data science objective is to design and evaluate a complete Retrieval-Augmented Generation pipeline capable of retrieving relevant legal articles and generating grounded compliance assessments for real estate contracts. The pipeline consists of three stages: document ingestion and article-level chunking, NLP enrichment with multilingual embedding and vector indexing, and semantic retrieval with reranking followed by LLM-based answer generation. The retrieval stage is benchmarked using $\text{Precision}@k$, $\text{Recall}@k$, MRR, and $\text{NDCG}@k$; the generation stage is evaluated using LLM-as-a-judge faithfulness and relevancy scoring.

3.9.3 Modeling Approach

3.9.3.1 Data Collection and Chunking

Legal articles were collected from two complementary sources. Articles from the Code des Obligations et des Contrats, the Code des Droits Réels, and the Code de l'Aménagement du Territoire et de l'Urbanisme were scraped article by article from the `9anoun.tn` web platform using `requests` and `BeautifulSoup`. Each scraped article was saved as a structured JSONL record.

For PDF-based legal codes, a document conversion pipeline was built using **Docling** with adaptive strategy selection: standard text extraction for digital PDFs and full-page OCR for scanned documents. Structural markers (Article, Titre, Chapitre, Section, Livre) were used as chunk boundaries. The resulting dataset comprised 1,116 article-level chunks across three legal codes.

3.9.3.2 NLP Enrichment — Keyword Extraction

Each article chunk was enriched using **KeyBERT** with the `Lajavaness/sentence-camembert-large` encoder, selected for its state-of-the-art performance on French semantic similarity tasks (Pearson correlation 85.9, Spearman correlation 85.8). CamemBERT was preferred over BERT for its

superior performance on French-language legal content. Keyword extraction was configured with n -gram range (1, 4), MMR diversity $\lambda = 0.3$, and `top_n=5` candidates per article.

3.9.3.3 Embedding and Vector Indexing

Each enriched article was embedded using **Cohere's** `embed-multilingual-v3.0`, selected for its multilingual support, standalone reranking capability, and strong performance in nearest-neighbor RAG retrieval. Vectors were indexed in **Qdrant Cloud** with cosine similarity, in a collection named `tunisian_laws` (1,024-dimensional vectors, 1,116 points).

3.9.3.4 Retrieval with Reranking

At query time, the top-10 candidate articles were retrieved from Qdrant by cosine similarity. The 10 candidates were then passed to Cohere's `rerank-multilingual-v3.0` cross-encoder, which re-scored each passage against the query using full attention over both texts. The top-5 results by reranking relevance score were returned.

3.9.3.5 Retrieval Evaluation

The retrieval pipeline was evaluated on a manually constructed test dataset of annotated query-ground-truth pairs. Four metrics were computed at $k = 5$:

$$\text{Precision@}k = \frac{|\text{Retrieved}_k \cap \text{Relevant}|}{k} \quad (3.2)$$

$$\text{Recall@}k = \frac{|\text{Retrieved}_k \cap \text{Relevant}|}{|\text{Relevant}|} \quad (3.3)$$

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q^{\text{first relevant}}} \quad (3.4)$$

$$\text{NDCG@}k = \frac{\text{DCG@}k}{\text{IDCG@}k}, \quad \text{DCG@}k = \sum_{i=1}^k \frac{\mathbb{I}[\text{doc}_i \in \text{Relevant}]}{\log_2(i+1)} \quad (3.5)$$

3.9.3.6 Answer Generation

Two large language models were evaluated as generators via the OpenRouter API:

- **GPT-OSS 120B** (openai/gpt-oss-120b:free)
- **LLaMA 3.1 70B Instruct** (hosted_vllm/Llama-3.1-70B-Instruct)

Both models operated under a system prompt instructing them to answer exclusively from the provided legal context, cite article numbers and source laws, and explicitly state uncertainty when the context is insufficient. Generation temperature was set to 0.1.

3.9.3.7 Generation Evaluation — LLM-as-a-Judge

The generation quality was evaluated automatically using an LLM judge on two axes: **Faithfulness** (whether the answer is exclusively grounded in the retrieved context) and **Answer Relevancy** (whether the answer directly addresses the user's query). Each metric was scored on a continuous scale from 0.0 to 1.0, aggregated over 20 test queries.

3.9.4 Implementation Overview

The selected technology stack combines:

- **Docling** for document ingestion (with OCR via Tesseract/EasyOCR)
- **CamemBERT** (Lajavaness/sentence-camembert-large) for keyword extraction
- **Cohere** embed-multilingual-v3.0 and rerank-multilingual-v3.0 for embedding and reranking
- **Qdrant Cloud** for vector storage and retrieval
- **Meta LLaMA** (hosted_vllm/Llama-3.1-70B-Instruct) for answer generation

The pipeline was implemented across two Jupyter notebooks: `chunking.ipynb` for data collection and chunking, and `embedder_retriever.ipynb` for embedding, indexing, retrieval, and end-to-end RAG evaluation. The final enriched corpus comprised 1,116 article-level chunks stored in `scraped_enriched_data.jsonl`.

3.10 BO8: Recommendation System

3.10.1 Business and Data Science Objective

The business objective of this module is to enhance user engagement through personalized property recommendations. The system aims to reduce information asymmetry in the Tunisian real estate market by providing users with relevant property suggestions based on natural language queries, rather than requiring rigid structured filters.

The data science objective is to develop a content-based recommendation system using user interactions and property similarity. From a data science perspective, the system focuses on:

- Extracting structured information from unstructured user queries
- Applying business constraints such as contract type, location, and budget
- Ranking properties based on their relevance to the user's intent
- Providing explainable recommendations to increase user trust

3.10.2 Justification of the Approach

The recommendation system is based on a rule-based and scoring-driven approach rather than a machine learning or deep learning model. This choice was motivated by several factors:

- The lack of labeled training data suitable for supervised learning
- The need for high explainability in real estate decisions

- The requirement to control business constraints explicitly (e.g., contract type, budget)
- The desire to ensure deterministic and stable behavior

Unlike black-box models, this approach allows full transparency in how recommendations are generated and ranked. The system is designed to be modular, allowing future integration of machine learning or LLM components.

3.10.3 System Architecture Overview

The recommendation system follows a modular pipeline architecture composed of six sequential components:

1. Query Classification
2. Query Understanding
3. Business Filtering
4. Ranking Engine
5. Explainability Module
6. Response Generation

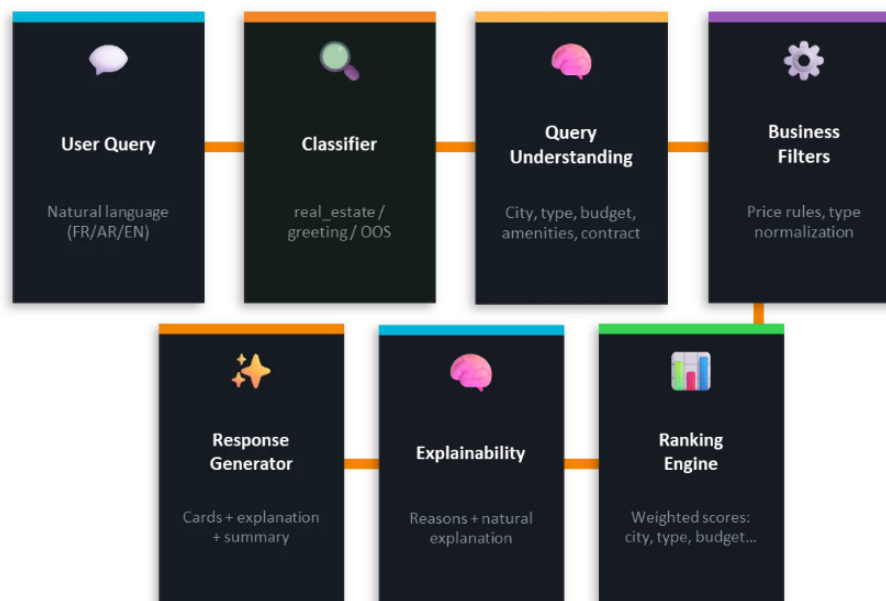


Figure 3.32: Recommendation system pipeline architecture: from natural language query to ranked property suggestions with explanations.

3.10.4 Query Classification

The first step classifies the user query into predefined categories: real estate query, greeting, out-of-scope query, or unknown. Classification is performed using keyword-based matching combined with heuristic rules, allowing the system to filter irrelevant queries and provide appropriate responses.

3.10.5 Query Understanding

The query understanding module extracts structured information from the user query, including city, property type, contract type (sale, rent, vacation), budget constraints, number of rooms, surface area, and amenities. This transforms unstructured text into a structured user intent representation.

3.10.6 Business Filters

After extracting the user intent, business filters are applied to reduce the candidate dataset, covering contract filtering, location filtering, budget constraints, and property type filtering.

3.10.7 Ranking Engine

The ranking engine assigns a score to each property using a weighted sum over multiple criteria: contract match, location match, property type match, budget compatibility, and additional features (rooms, surface, amenities). Properties are sorted in descending order of their score.

3.10.8 Explainability

Each recommendation is accompanied by an explanation highlighting matching criteria, partially matched constraints, and additional relevant features. This increases user trust and transparency.

3.10.9 Response Generator

The response generator converts the system output into a user-friendly message containing a summary, a list of recommended properties, and explanations for each result.

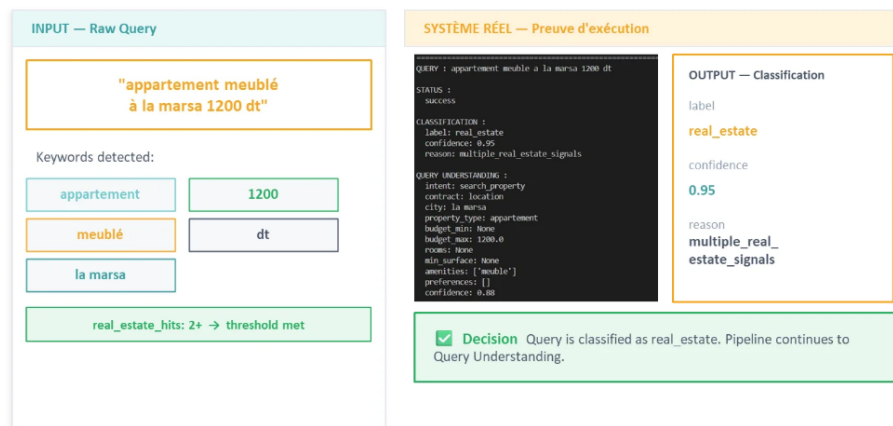


Figure 3.33: Recommendation system results: example query processing and ranked property output (1/2).



Figure 3.34: Recommendation system results: explainability output and system robustness examples (2/2).

3.10.10 Recommendation Strategy and Evaluation

The system follows a hybrid strategy combining rule-based filtering and scoring-based ranking, prioritizing exact matches and applying controlled constraint relaxation when no exact match is found. Evaluation was performed qualitatively using real user queries, assessing relevance of returned results, consistency of explanations, and system robustness to different query formulations.

3.11 Conclusion

Eight business objectives were addressed in this modeling chapter, covering the full pipeline from ETL and data enrichment to price prediction, anomaly detection, market trend analysis, legal compliance, and recommendation.

- **BO1** established a scalable, automated ETL pipeline from raw Tayara listings to structured PostgreSQL tables.
- **BO2** implemented a two-branch NLP and Computer Vision extraction pipeline for recovering structured attributes from unstructured text and images.
- **BO3** built a RAG system grounded in large-scale statistical and economic documents, combining Docling ingestion, CamemBERT keyword extraction, Cohere embedding, Qdrant retrieval, and LLaMA generation.
- **BO4** selected Random Forest (tuned) as the best price prediction model, achieving MAE of 32,435 TND and R^2 of 0.848, with SHAP-based explainability.
- **BO5** deployed a consensus-based anomaly detection pipeline combining Z-score, IQR fence, and Isolation Forest, with opportunity labelling and geospatial visualization.
- **BO6** analyzed regional price trends using time-series forecasting models including ARIMA, Exponential Smoothing, and Prophet.
- **BO7** developed a legal compliance RAG system over 1,116 Tunisian legal article chunks, evaluated on faithfulness and relevancy using an LLM-as-a-judge protocol.
- **BO8** designed a modular, explainable content-based recommendation system for personalized property suggestions from natural language queries.

Each module was built to be modular, explainable, and production-ready, collectively transforming a fragmented real estate market into an intelligent decision-support platform.

Deployment

4.1 Introduction

This chapter presents the deployment phase of the EstateMind project, following the completion of the modeling and analytical pipeline described in the previous chapter. Deployment constitutes the final stage of the TDSP lifecycle, in which the developed models, agents, and data services are integrated into a functional, accessible, and production-oriented platform.

The EstateMind deployment is organized as a multi-service architecture combining a public-facing web application, an internal administration dashboard, a REST API backend, and a set of specialized AI microservices. Each component is containerized and communicates through well-defined interfaces, ensuring modularity, maintainability, and scalability.

This chapter describes the global architecture of the deployed system, the technical stack adopted for each component, the AI agent integration layer, the key engineering decisions made during deployment, and the access control and security mechanisms implemented.

4.2 Global Architecture

The deployed EstateMind platform consists of nine distinct services, each fulfilling a specific role within the overall system. Table 4.1 presents the full service inventory with the corresponding technology, network port, and functional role of each component.

Table 4.1: EstateMind deployed service architecture.

Component	Technology	Port	Role
Admin Dashboard	React 18 + Vite	5173	Internal analytics and data management interface
Public Site	Next.js 15 (App Router)	3001	End-user facing application: search, AI agents, legal analysis
Backend API	FastAPI + SQLite	3002	Data layer, authentication, statistics endpoints
Law Agent	Docker (FastAPI)	8002	RAG-based legal assistant (Qdrant + OpenRouter)
Market Agent	Docker (FastAPI)	8001	Market pocket detection and RAG comparables
Price Agent	Docker (FastAPI)	8003	ML-based property price prediction
Anomaly Agent	Docker (FastAPI)	8004	Listing anomaly and mispricing detection
Captioner	Docker (FastAPI + GPU)	8000	BLIP-2 image captioning for property photos
Recommender	Render (cloud)	—	NLP-based semantic property recommendation

The architecture follows a clear separation of concerns: the public site and admin dashboard constitute the presentation layer; the backend API handles data persistence and authentication; and the AI microservices operate as independent inference endpoints consumed by the frontend through proxied API routes.

Figure 4.1 illustrates the global deployment architecture and the communication flows between components.

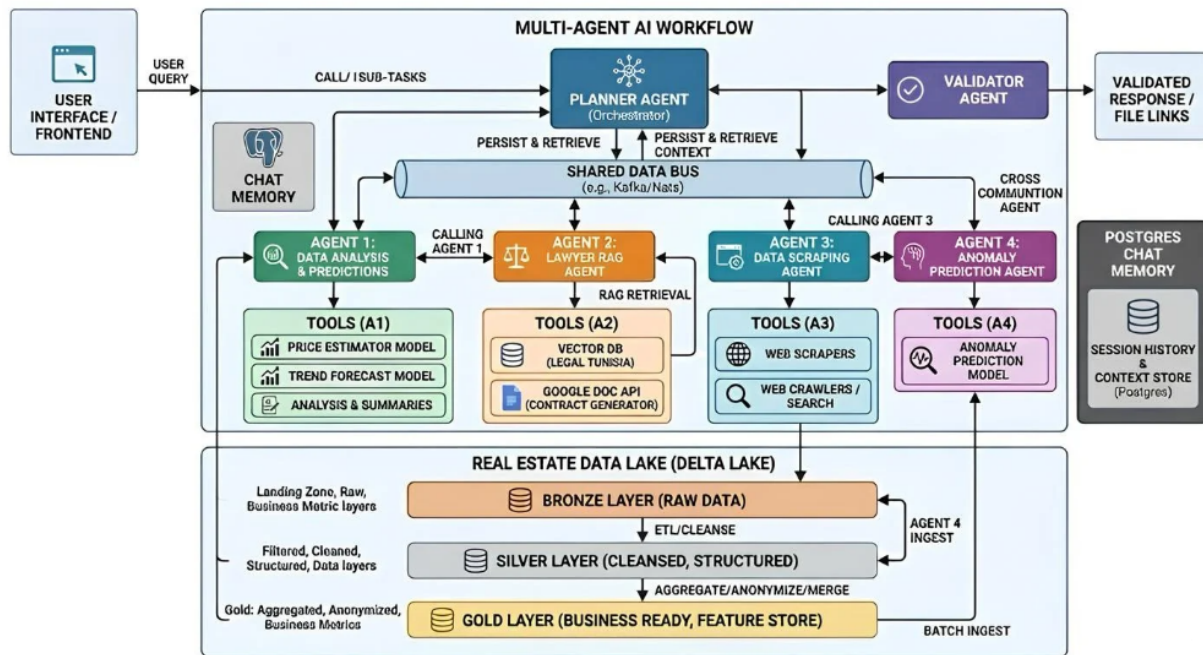


Figure 4.1: Global deployment architecture of the EstateMind platform, showing service interactions and communication flows.

4.3 Public-Facing Application

4.3.1 Technology Stack

The public site was developed using **Next.js 15** with the App Router paradigm, providing server-side rendering, file-based routing, and built-in API route proxying. The application is accessible at port 3001 and serves as the primary interface through which end users interact with EstateMind’s core features: property search, AI-powered price estimation, anomaly detection, legal contract analysis, and personalized property recommendations.

Authentication is handled through **NextAuth.js**, using a JWT-based session strategy. User roles are stored directly in the JWT token, enabling lightweight access control without a dedicated authorization service.

4.3.2 AI Agent Integration

The public site integrates five specialized AI agents, each exposed through a dedicated Next.js API proxy route. This design prevents direct client-to-agent communication, allowing timeout management, error handling, and authentication to be centralized at the proxy layer.

4.3.2.1 Price Estimation Agent

The price agent (port 8003) exposes a FastAPI endpoint that receives a property feature dictionary and returns a predicted price in Tunisian Dinar. The public site collects user inputs through a structured form and forwards them to the agent through the `/api/chat/price` proxy route. The response includes the predicted price and a confidence interval derived from the model's MAE.

4.3.2.2 Anomaly Detection Agent

The anomaly agent (port 8004) receives a listing identifier or feature set and returns an anomaly flag, a five-tier opportunity label (from Strong Opportunity to Overpriced Risk), and a composite confidence score. This agent is consumed both through the property detail pages and through a dedicated anomaly explorer interface in the public site.

4.3.2.3 Law Agent

The law agent (port 8002) implements the RAG pipeline described in the modeling chapter. It accepts a natural language legal query, retrieves the most relevant articles from the Qdrant vector store using Cohere embeddings, and generates a sourced answer via the OpenRouter LLM API.

Two integration issues were identified and resolved during deployment. First, the agent response payload uses the field `data.result`, whereas both the chat widget and the legal analysis page were initially reading `data.answer`, causing all responses to appear as errors. This was corrected by updating the frontend response parser to check `data.result` first.

Second, the RAG pipeline — combining vector search and LLM generation — requires between 60 and 120 seconds per query. The proxy timeout was increased from 90 seconds to 240 seconds across three locations: the Next.js proxy route, the chat widget `fetch()` call, and the legal analysis page `fetch()` call.

4.3.2.4 Legal Analysis Result Presentation

The raw markdown output produced by the law agent was restructured into a custom rendering layer to improve readability for non-technical users. The renderer:

- Parses the markdown into sections delimited by `##` headings
- Renders each section as a collapsible card with a contextual icon (shield for risks, clipboard for recommendations)
- Color-codes risk table rows — red for critical issues involving nullity or fraud keywords, orange for moderate risks
- Presents recommendations as sequentially numbered steps rather than bullet points
- Displays a summary bar at the top of the page showing total risk count, total action count, and a copy-to-clipboard button
- Opens the first two sections by default, collapsing the remainder to avoid information overload

Figure 4.2 shows the legal analysis result page with the custom rendering layer applied.

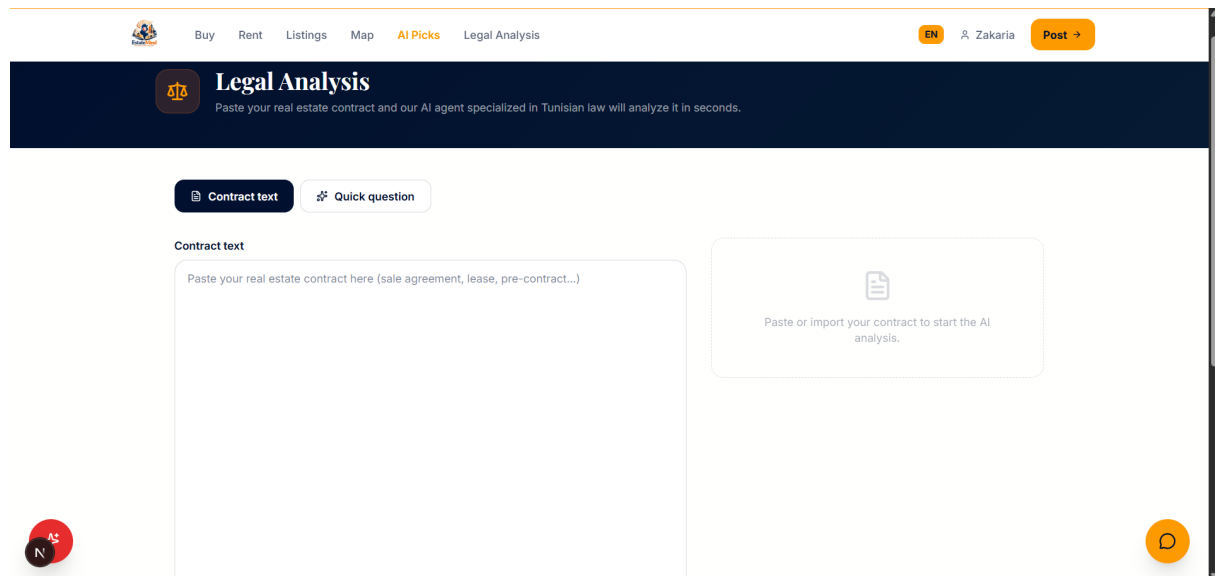


Figure 4.2: Legal Analysis page of the EstateMind public site, allowing users to paste a real estate contract for AI-powered analysis grounded in Tunisian legal codes.

4.3.2.5 Recommendation Agent

The recommendation agent is hosted externally on **Render** (cloud, free tier) as a stateless NLP service. It receives a natural language property query, extracts structured intent (city, property type, contract type, number of rooms, budget), and returns a ranked list of matching properties.

Given the free-tier cold start latency of up to 60 seconds, a local fallback mechanism was implemented: if the Render service returns an empty result set, the Next.js proxy route automatically re-queries the local backend API using the parsed intent filters and a service authentication token, ensuring that users always receive real listings regardless of the external service state.

The recommendation widget displays NLP-extracted intent chips (city, type, rooms, budget), listing cards with key attributes, and a direct link to the search page with pre-filled filters corresponding to the extracted intent.

Figure 4.3 shows the recommendation widget in the public interface.

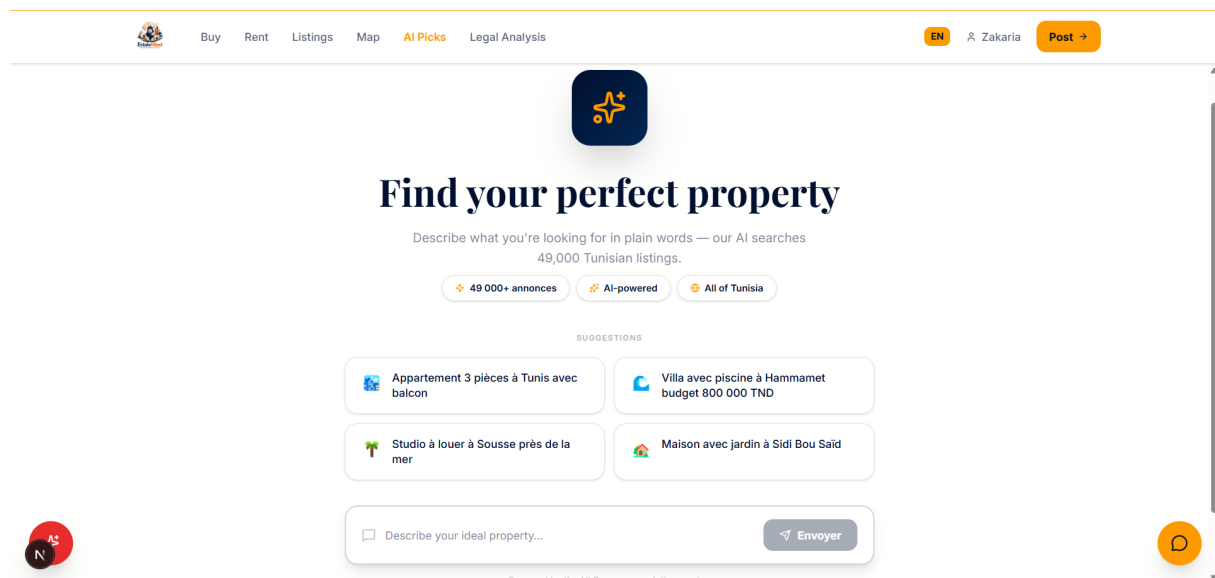


Figure 4.3: Property recommendation widget showing NLP-extracted intent chips and ranked listing cards.

4.3.2.6 Image Captioning Service

The captioner service (port 8000) runs a **BLIP-2** image captioning model on GPU and is called when a property image is submitted for automated description generation. This service enriches listings that lack textual descriptions by generating natural language captions from property photographs.

4.4 Administration Dashboard

4.4.1 Overview

The administration dashboard was developed using **React 18** with **Vite** as the build tool, accessible at port 5173. It provides internal team members with access to platform analytics, data quality monitoring, scraping pipeline status, and listing management. Access is restricted to users with the admin role, verified through the JWT session token issued by the backend.

4.4.2 Internationalization

The dashboard supports three languages — French, Arabic, and English — through a centralized translation dictionary (`i18n.js`) exposed via a React context (`LangContext`). All interface strings across the Listings, Scrapers, and PowerBI pages are resolved through this context using a `useLang()` hook. Arabic activates a right-to-left layout automatically upon language selection.

4.4.3 Interactive Map

A Tunisia-wide property map was integrated into the dashboard, capable of rendering over 20,000 listing locations simultaneously. To handle this volume without browser degradation, the map uses **Leaflet's canvas renderer** (`L.canvas()`), which draws all markers in a single canvas draw call rather than inserting individual SVG DOM nodes. The 16,000+ geo-validated listings are fetched once at load and cached in memory; the interface displays a live count during data retrieval.

Figure 4.4 shows the interactive listing map with 20,000 rendered points.

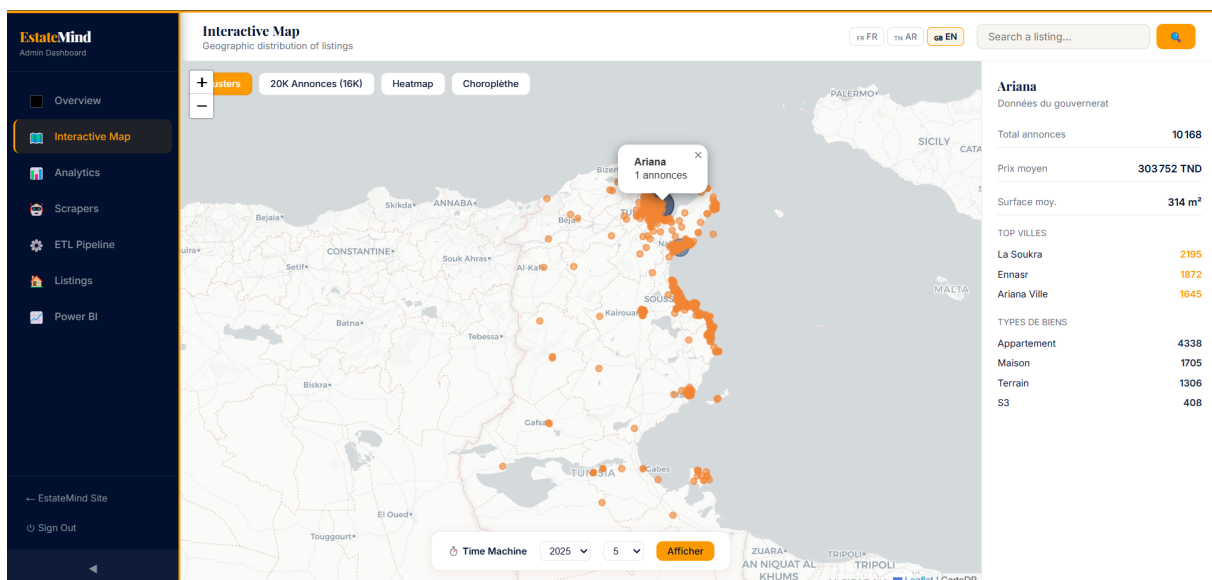


Figure 4.4: Interactive Tunisia map in the administration dashboard, rendering over 20,000 listing locations using Leaflet canvas.

4.4.4 Property Image Handling

A `PropertyImage` component was developed to handle the high rate of missing or broken image URLs common in scraped listing data. The component renders the actual scraped image when available, and falls back to an inline SVG house illustration labeled with the property type when the image URL is absent or returns an error. The component resets its error state whenever the source URL changes, ensuring correct behavior during gallery navigation in the listing detail modal.

4.5 Backend API

4.5.1 Overview

The backend API was implemented using **FastAPI** with **SQLite** as the persistence layer, accessible at port 3002. SQLite was selected over a client-server database such as PostgreSQL because the dataset of approximately 49,000 listings fits entirely in memory, eliminating network overhead and simplifying the deployment environment.

4.5.2 Performance Optimizations

Several performance issues were identified during integration testing and resolved before final deployment.

4.5.2.1 Database Indexing

Seven SQLite indexes are created automatically at server startup, covering the most frequently queried columns: `gouvernerat`, `type`, `contrat`, `prix`, `surface`, `latitude/longitude`, and `date_publication`. This optimization reduced statistics endpoint response times from a range of 2–6 seconds to under 100 milliseconds.

4.5.2.2 Cache Warming

A background thread is launched at boot to pre-fill the in-memory response cache for all computationally expensive endpoints (`/stats/*`, `/map/*`, `/pipeline/quality`). This ensures that the first user request always hits a warm cache rather than triggering a cold query on the full dataset.

4.5.2.3 Query Optimization

Four endpoint-level query inefficiencies were identified and corrected:

- `/stats/price-distribution`: float division `prix/50000` was generating over 4,000 histogram bins. Fixed by casting to integer: `CAST(prix/50000 AS INTEGER)*50000`.
- `/stats/insights`: a correlated subquery was executing per row across 200,000 records. Replaced with a CTE that pre-computes averages once.
- `/stats/amenities`: ten separate `COUNT(*)` queries were replaced with a single query using ten `SUM(CASE WHEN)` aggregations.
- `/pipeline/quality`: eight separate queries totalling 6,300ms were consolidated into a single `SUM(CASE WHEN)` query.

4.5.3 Service Authentication

A static `SERVICE_TOKEN` environment variable was added to the backend authentication middleware. Internal server-to-server calls — specifically from the Next.js recommendation proxy route to the backend fallback endpoint — use this token to bypass the session-based login flow without exposing user credentials or requiring a dedicated service account.

4.6 Access Control and Security

4.6.1 Role-Based Access

A dedicated administrator account (`admin@estatemind.tn`, `role: admin`) was created in the user database. The NextAuth JWT callback stores the user role in the session token at login. Both the dark-themed homepage navbar and the light-themed inner-page navbar check `session.user.role === "admin"` before rendering the link to the administration dashboard. Mobile navigation menus apply the same role check. This ensures that the internal dashboard URL is not surfaced to unauthenticated or non-administrator users.

4.6.2 Environment Isolation

Each AI microservice runs in an isolated Docker container with its own dependency environment. Inter-service communication uses internal Docker network addresses, and only the ports listed in Table 4.1 are exposed to the host. The Qdrant vector store and Cohere API credentials are passed as environment variables and are never hardcoded in source files.

4.7 Key Engineering Decisions

Table 4.2 summarizes the principal engineering decisions made during the deployment phase and the rationale behind each choice.

Table 4.2: Key engineering decisions made during the EstateMind deployment phase.

Decision	Rationale
SQLite for the admin backend	49,000 rows fits in memory; eliminates PostgreSQL server overhead in a single-node deployment
Canvas renderer for map markers	SVG cannot handle 16,000+ DOM nodes; canvas renders all points in a single draw call
Render for the recommender	Free-tier hosting for a stateless NLP service; cold starts are acceptable for non-critical queries
Local fallback for recommendations	Render’s dataset may diverge from the local database; fallback guarantees users always see real listings
240-second law agent timeout	RAG pipeline (vector search + LLM generation) requires 60–120 seconds; premature timeouts cause silent failures
JWT role claim for admin gating	Simple string check on an existing token; no separate authorization service required
Service token for internal calls	Decouples server-to-server authentication from user session management without exposing credentials

4.8 Conclusion

This chapter presented the deployment architecture of the EstateMind platform. A multi-service system was designed and implemented, integrating a public-facing Next.js application, a React-based administration dashboard, a FastAPI backend, and five specialized AI microservices deployed as Docker containers or cloud-hosted endpoints. Key engineering challenges addressed during this phase include query performance optimization, canvas-based high-density map rendering, RAG pipeline timeout management, multi-language interface support, and role-based access control. The resulting deployment constitutes a functional and production-oriented platform that operationalizes the analytical models developed throughout the project.



GENERAL CONCLUSION

The EstateMind project set out to address a fundamental challenge: the Tunisian real estate market, despite its economic significance, operates largely in the dark. Buyers overpay. Sellers undersell. Investors navigate without reliable signals. Professionals rely on intuition where data should lead.

Over the course of this project, we built a response to that challenge — not a single model, but an integrated intelligence platform spanning the full data science lifecycle. From multi-source data acquisition and ETL engineering to NLP extraction, price prediction, anomaly detection, market trend analysis, legal compliance, recommendation, and production deployment, each component was designed to be modular, explainable, and production-ready.

The key contributions of this work can be summarized as follows. A scalable ETL pipeline was developed to unify fragmented real estate listings from eleven Tunisian platforms into a single, structured analytical repository. A two-branch NLP and Computer Vision pipeline was built to recover structured insights from unstructured listing descriptions and property images. A RAG system was designed to surface relevant insights from large-scale statistical documents and Tunisian legal codes, enabling both market intelligence and contract compliance analysis. A price estimation engine based on Random Forest regression achieved a Mean Absolute Error of 32,435 TND and an R^2 of 0.848, providing data-backed valuations for the first time in this market context. A consensus-based anomaly detection pipeline combining Z-score, IQR fence, and Isolation Forest was deployed to flag underpriced and overpriced properties with opportunity labels and confidence scores. Regional price trends were modeled using time-series forecasting methods including ARIMA, Exponential Smoothing, and Prophet. A content-based recommendation system was implemented to match users to properties based on

natural language queries. Finally, all modules were integrated into a multi-service production platform combining a public-facing Next.js application, a React administration dashboard, a FastAPI backend, and five specialized AI microservices.

Beyond its technical contributions, EstateMind carries a broader ambition: to reduce information asymmetry in a market where opacity has long favored the few over the many. By aligning with United Nations Sustainable Development Goals 9, 10, 11, and 12, this project demonstrates that data science can serve not only business efficiency, but also market fairness and social equity.

Several limitations were encountered throughout this work. The dataset relies exclusively on asking prices rather than finalized transaction values, geographic coverage is biased toward major urban governorates, and the absence of labeled ground truth in an informal market posed ongoing evaluation challenges. These limitations define clear directions for future work.

Looking ahead, EstateMind can be extended along several axes. On the data side, integration of official transaction registries and real-time scraping pipelines would significantly improve model accuracy and temporal coverage. On the modeling side, incorporating user behavioral data would enable a fully personalized recommendation engine. On the deployment side, a mobile application and an API marketplace targeting real estate agencies would accelerate adoption. On the legal side, automated contract generation grounded in Tunisian law represents a natural next step for the compliance module.

EstateMind is not the finished product — it is the foundation. The Tunisian real estate market is ready for this transition. The data exists. The models are trained. The platform is live. What comes next is scale.

APPENDIX: LinkedIn Posts

In accordance with the project defense requirements, each team member published a LinkedIn post presenting the EstateMind project. The following section compiles the direct links and screenshots of each post.

Sirine Hjaij

Post URL: <https://www.linkedin.com/posts/...>

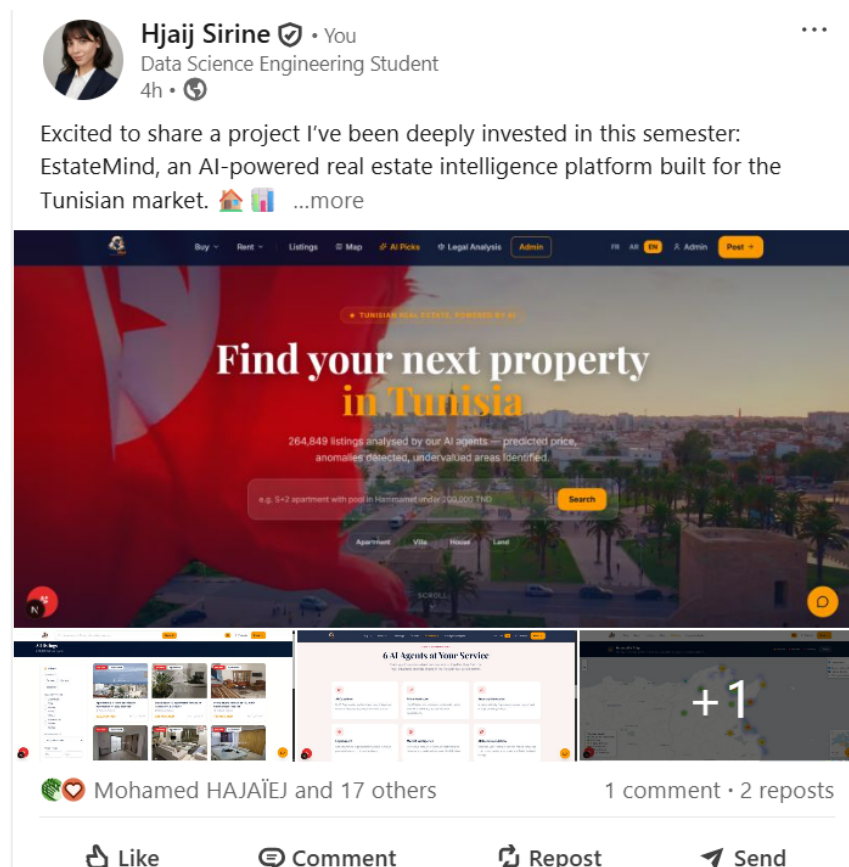


Figure 4.5: LinkedIn post — Sirine Hjaij

Farah Ben Yedder

Post URL: https://www.linkedin.com/posts/farah-ben-yedder_integratedproject-technologies-datascience-ugc?utm_source=share&utm_medium=member_desktop&rcm=ACoAACUXvc4BnTUghQ1e0yt3Pj0pCS0gaie1E

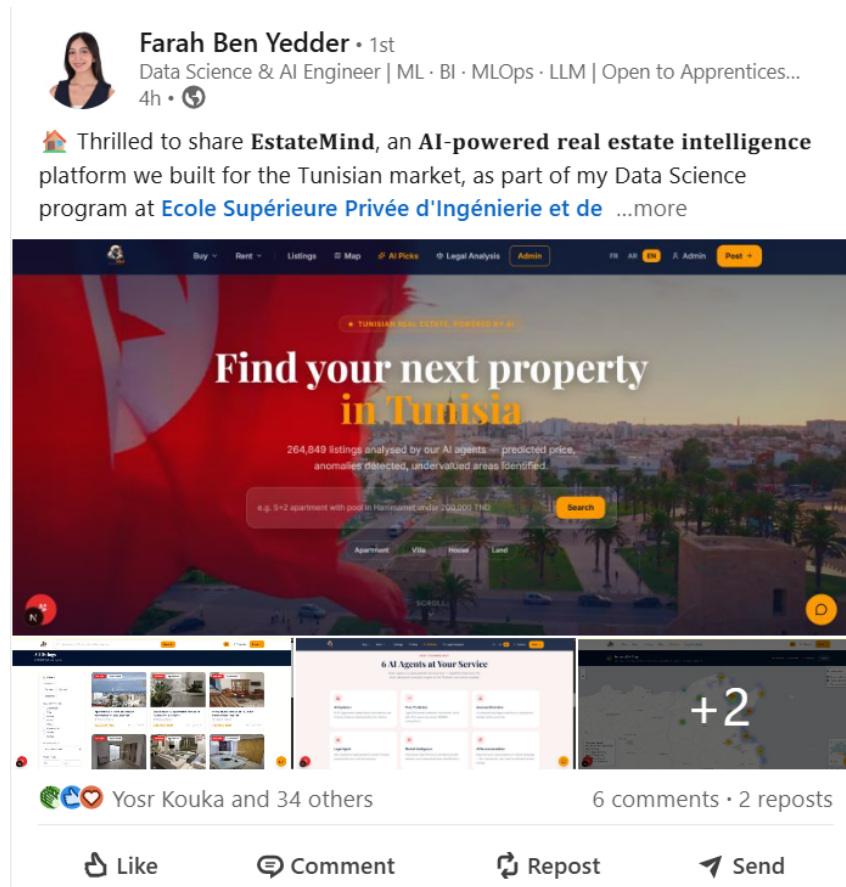


Figure 4.6: LinkedIn post — Farah Ben Yedder

Zakaria Bouchaddakh

Post URL: https://www.linkedin.com/posts/zakaria-bouchaddakh-074876276_artificialintelligence-datascien?utm_source=share&utm_medium=member_desktop&rcm=ACoAACUXvc4BnTUghQ1e0yt3Pj0pCS0gaie1E

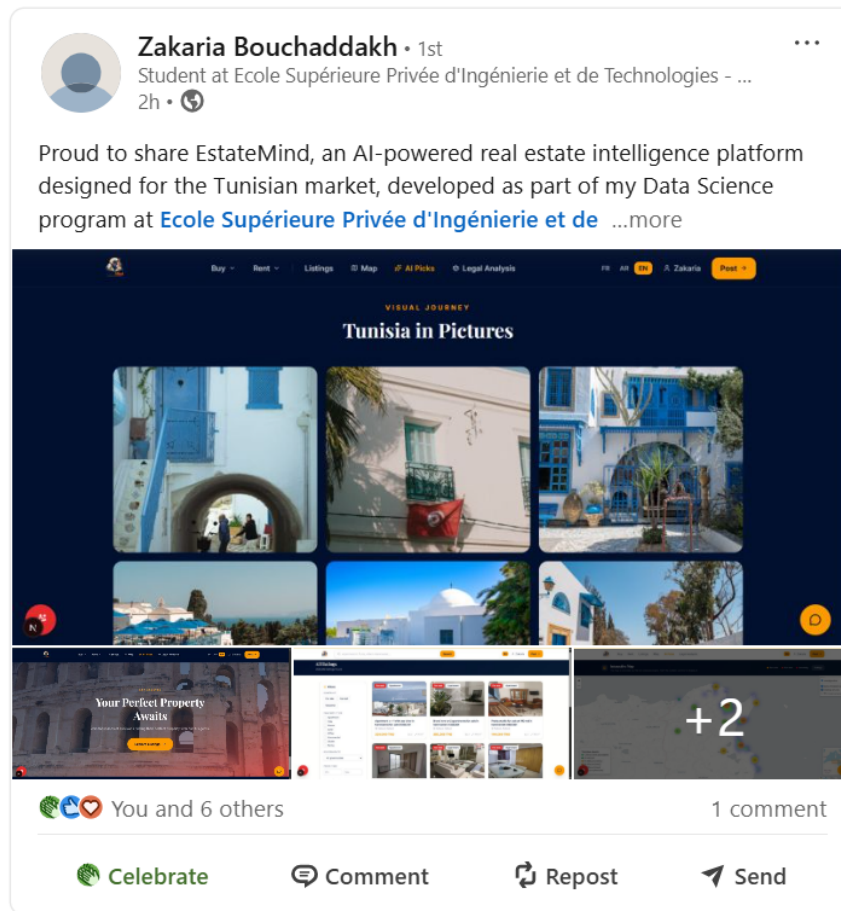


Figure 4.7: LinkedIn post — Zakaria Bouchaddakh

Wael Ben Salem

Post URL: https://www.linkedin.com/posts/wael-ben-salem-299060237_integratedproject-artificialintelligenceutm_source=share&utm_medium=member_desktop&rcm=ACoAACUXvc4BnTUghQ1e0yt3Pj0pCS0gaie1E

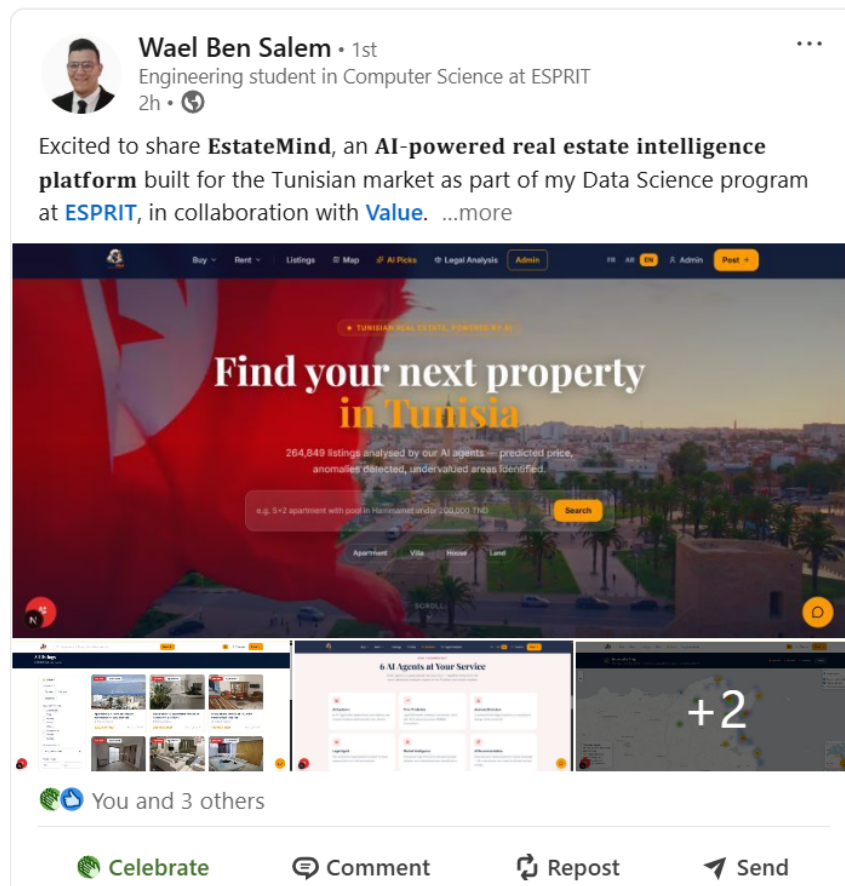


Figure 4.8: LinkedIn post — Wael Ben Salem

Mohamed Abdennadher •

Post URL: https://www.linkedin.com/posts/mohamed-abdennadher-37b116278_integratedproject-datascience-9?utm_source=share&utm_medium=member_desktop&rcm=ACoAACUXvc4BnTUghQ1e0yt3Pj0pCS0gaie

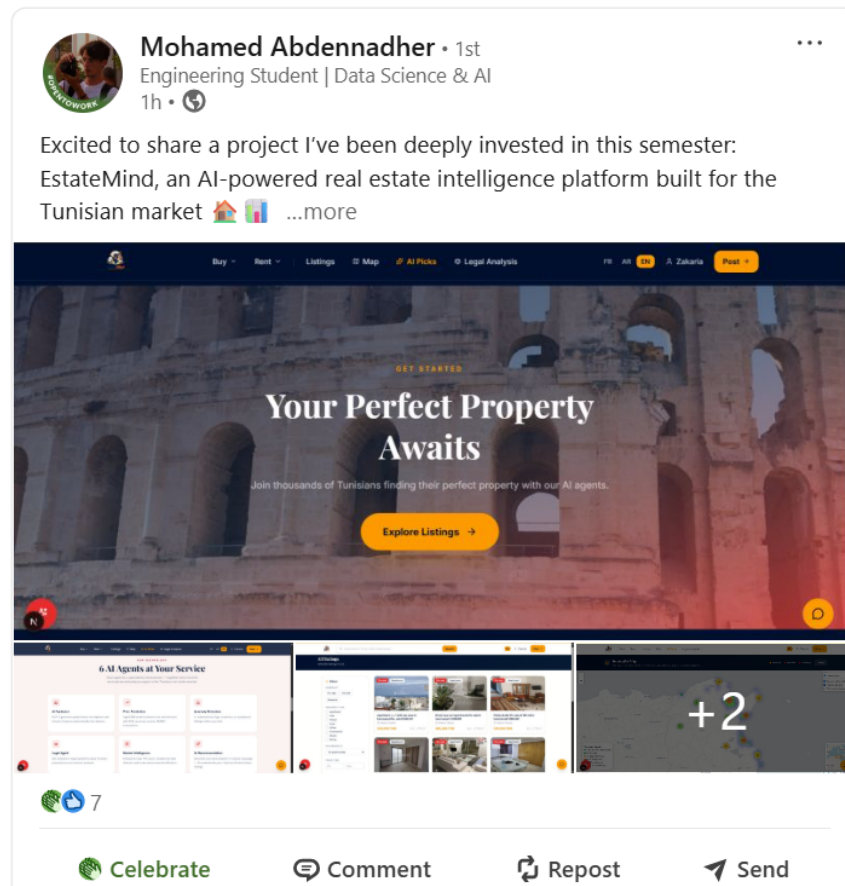


Figure 4.9: LinkedIn post — Mohamed Abdennadher



BIBLIOGRAPHY

- [1] Microsoft. *Team Data Science Process (TDSP)*. Available at:
<https://learn.microsoft.com/en-us/azure/architecture/data-science-process/>
- [2] United Nations. *Sustainable Development Goals*. Available at:
<https://sdgs.un.org/goals>
- [3] World Bank. *Housing Markets and Urban Development*. Available at:
<https://www.worldbank.org/en/topic/urbandevelopment>
- [4] OECD. *Artificial Intelligence and Data in Real Estate Markets*. Available at:
<https://www.oecd.org/digital/artificial-intelligence/>
- [5] Institut National de la Statistique (INS). *Statistical Data on Population and Socio-Economic Indicators in Tunisia*. Available at:
<http://www.ins.tn>
- [6] Hastie, T., Tibshirani, R., Friedman, J. *The Elements of Statistical Learning*. Springer, 2009.
- [7] Chapman, P., Clinton, J., Kerber, R., et al. *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS, 2000. Available at:
<https://www.the-modeling-agency.com/crisp-dm.pdf>
- [8] Batini, C., Cappiello, C., Francalanci, C., Maurino, A. *Methodologies for Data Quality Assessment and Improvement*. ACM Computing Surveys, 2009.
- [9] Mitchell, R. *Web Scraping with Python*. O'Reilly Media, 2018.
- [10] Little, R.J.A., Rubin, D.B. *Statistical Analysis with Missing Data*. Wiley, 2002.

